REGRET IN RUNTIME ANALYSIS OF BLACK-BOX ALGORITHMS

A report submitted to The University of Manchester
for the degree of Bachelor of Science
in the Faculty of Science and Engineering

2026

Manoj Manikandan

Supervised by Dr. Jonathan Shapiro

Department of Computer Science

Contents

Abstract **8**

Declaration **9**

Acknowledgements **10**

Abbreviations & Acronyms **11**

1 Introduction **13**

1.1 Motivation 13

1.1.1 Conventional Solutions 13

1.1.2 The Problem 14

1.1.3 Project's Alternative 14

1.2 Objectives 15

1.2.1 General Aims 15

1.2.2 Theoretical Exploration 15

1.2.3 Supplementary Software Development 15

1.2.4 Experimental Analysis 15

1.3 Project Outline 16

1.3.1 Literature Study 16

1.3.2 Isolating Relevant Benchmarks 16

1.3.3 Development of Experiment framework 16

1.3.4 Orchestrating Experiments 16

1.3.5 Evaluating Results 16

1.3.6 16

1.4 Ethical Considerations 17

1.5 Report Structure 18

1.5.1 Background 18

1.5.2 Software Design 18
1.5.3 Experiment Structure 18
1.5.4 Evaluation & Discussion of Results 18
1.5.5 Conclusion & Future Work 18

2 Background **19**
2.1 Computational Problems 19
2.1.1 Problem Formulations 19
2.1.2 Algorithms 20
2.1.2.1 Algorithm Properties 20
2.1.2.2 Oracle-Based Computation 21
2.1.3 Search Space 21
2.1.3.1 Structure of the Search Space 21
2.1.3.2 Constraints and Feasibility 22
2.1.4 Objective Functions and Landscape Properties 22
2.1.4.1 Zeroth-, First-, and Second-Order Methods . . . 23
2.1.4.2 Landscape Properties 24
2.1.5 Complexity Measures 24
2.1.5.1 Classical Complexity: Time and Space 24
2.1.5.2 Information Complexity: Query and Oracle Complexity 24
2.1.5.3 Expected vs. Worst-Case Analysis 25
2.2 Black-Box Optimization 26
2.2.1 The Black-Box Model 26
2.2.1.1 Variants of Black-Box Models 26
2.2.2 Black-Box Complexity 26
2.2.3 Purpose of Randomized Search Heuristics 26
2.2.3.1 No-Free-Lunch Theorems 26
2.2.3.2 Taxonomy of Derivative-Free Methods 26
2.2.4 Randomized Search Heuristic Algorithms 26
2.2.4.1 Evolutionary Algorithms (EA) 26
2.2.4.2 Simulated Annealing 27
2.2.4.3 Randomized Local Search (RLS) 29
2.2.5 Few Pseudo-Boolean Problems 29
2.2.5.1 OneMax 29
2.2.5.2 LeadingOnes 30

2.2.5.3 Jump 30
2.2.5.4 BinaryValue[BinVal] 31
2.2.5.5 Trap 31
2.2.5.6 Plateau 31
2.2.5.7 Hierarchical If-and-Only-If (HIFF) 32
2.2.5.8 MaxSAT 33
2.2.5.9 Tunable Landscapes: NK-Landscapes 34
2.2.6 Traditional Performance Measures in BBO 34
2.2.6.1 Expected Runtime (Hitting Time) 35
2.2.6.2 Fixed-Budget Analysis 35
2.2.6.3 Fixed-Target Analysis and Runtime Profiles . . . 35
2.2.6.4 Limitations and the Case for a Continuous Performance Measure 36
2.3 Sequential and Online Optimization Frameworks 38
2.3.1 Exploration-Exploitation Trade-off 38
2.3.2 Multi-Armed Bandits 38
2.3.3 Bayesian Optimization 38
2.4 Computational Regret 40
2.4.1 Definition of Regret 40
2.4.2 Types of Regret 40
2.4.2.1 Instantaneous Regret 40
2.4.2.2 Cumulative Regret 40
2.4.2.3 Simple Regret 41
2.4.2.4 Normalized Regret 42
2.4.3 Regret Bounds 42
2.4.4 Connections to Runtime Analysis 42
2.5 Regret-Aware Black-Box Optimization 43
2.5.1 Regret as a Performance Measure for Randomized Search Heuristics 43
2.5.2 Related Work: Regret in Bayesian and Surrogate-Based Optimization 43
2.5.3 Related Work: Regret-Inspired Analysis of Evolutionary Algorithms 43
2.5.4 Related Work: Bandit-Inspired Metaheuristics 43
2.5.5 Connections to Runtime Analysis via Drift Theory 43

2.5.6 Open Questions and Project Scope 43

3 Supplementary Software Design 44

3.1 Purpose of the Software 44

3.1.1 Preexisting software 45

3.1.2 Capabilities 45

3.1.3 Design Decisions 46

3.1.3.1 Independent Job Execution framework 46

3.1.3.2 Raw Output JSON files 47

3.1.3.3 UML Diagram 47

3.1.4 Limitations 48

3.1.5 Result Interpretations 48

3.2 Testing strategy 51

3.3 Project Dependencies 52

4 Runtime Profiles and Anytime Performance 54

4.1 Motivation 54

4.2 Variants 55

4.2.1 Expectation-Based Runtime Profiles 55

4.2.2 Distributional Runtime Profiles 55

4.3 Connection to Cumulative Regret 56

4.4 Practical Relevance 59

5 Experiments 61

5.1 Experiment Structure 61

5.1.1 Repetitions and budget 61

5.1.2 Execution mode 61

5.1.3 Visualization and Table Strategy 62

5.1.4 Suite Numbering and Roles 63

5.2 Baseline Suites 65

5.2.1 Suite 01: Algorithm-Family Comparison 65

5.2.2 Suite 02: Extended Pseudo-Boolean Benchmark 65

5.3 Verification Suite 66

5.3.1 Suite 08: Runtime Profile Verification 66

5.4 Exploratory Analysis of Regret and Runtime 67

5.4.1 Suite 03: NK Landscape Ruggedness 67

5.4.2 Suite 04: Trap Function Mutation Rate 67
5.4.3 Suite 05: Jump Function Mutation Rate 68
5.4.4 Suite 06: Plateau Function Mutation Rate 68
5.4.5 Suite 07: Hierarchical IFF Scaling 68
5.4.6 Suite 09: Jump Plateau Duration 69
5.4.7 Suite 10: Barrier Structure Comparison 69
5.4.8 Suite 11: SA Cooling Schedule Diagnostics 70
5.4.9 Suite 12: NK Saturation 70
5.4.10 Suite 13: Regret Scaling with Problem Size 71
5.4.11 Suite 14: Population Dynamics 71
5.5 Cross-Suite Design Relationships 72

6 Evaluation & Discussion of Results 74
6.1 Overview 75
6.2 Baseline Suites 75
6.2.1 Suite 01: Algorithm-Family Comparison 75
6.2.1.1 Aggregate Regret and Convergence 75
6.2.1.2 Regret Distributions and TTFO 75
6.2.1.3 Trajectory Analysis 75
6.2.2 Suite 02: Extended Pseudo-Boolean Benchmark 75
6.2.2.1 Aggregate Regret and Convergence 75
6.2.2.2 Effect of Algorithm Overrides 75
6.2.2.3 Trajectory Analysis 75
6.3 Verification Suite 75
6.3.1 Suite 08: Runtime Profile Verification 75
6.3.1.1 Tail-Sum Identity: Direct vs. Profile-Derived $\mathbb{E}[\mathrm{CR}(T)]$ 75
6.3.1.2 Inverse Runtime Profile Surfaces and Curves . . . 75
6.4 Exploratory Analysis 75
6.4.1 Barrier Structure and Mutation Rate 75
6.4.1.1 Suite 04: Trap – Deception Width and Mutation Rate 75
6.4.1.2 Suite 05: Jump – Mutation Rate and Plateau Onset 75
6.4.1.3 Suite 06: Plateau – Neutral Walk and SA 75
6.4.1.4 Suite 09: Jump – Plateau Duration 75
6.4.1.5 Suite 10: Barrier Archetype Comparison at k=5 . 75
6.4.2 NK Ruggedness and SA Cooling 75

6.4.2.1 Suite 03: Cooling Schedule × Epistasis Grid . . . 75
6.4.2.2 Suite 11: SA Cooling Schedule Diagnostics 75
6.4.2.3 Suite 12: NK Saturation 75
6.4.3 Hierarchical Structure and Scaling 75
6.4.3.1 Suite 07: HIFF Scaling with Problem Size 75
6.4.3.2 Suite 13: Regret Scaling with n on OneMax . . . 75
6.4.4 Population Dynamics 75
6.4.4.1 Suite 14: (mu+lambda)-EA vs. (1+1)-EA 75
6.5 Cross-Suite Synthesis 75
6.5.1 Regret Trajectory Shapes as Landscape Diagnostics 75
6.5.2 When TTFO Fails and Cumulative Regret Succeeds 75
6.5.3 Runtime Profiles and Regret: Complementary Views . . . 75

7 Conclusion & Future Work 76
7.1 Conclusion 76
7.2 Future Work 76

A Evolutionary Algorithms, Extended 83
A.1 Exact Runtime Formulae 83
A.2 Implementation-Aware Variants 83
A.3 General Form - '$(\mu+\lambda)$-EA' 84
A.4 Genetic Algorithms 84
A.4.1 Crossover 84
A.5 Parameters & Operators 84
A.5.1 Adaptive Mutation & crossover rates 84
A.5.2 Selection pressure 84

Word Count: 11142

Abstract

TODO: Add abstract after completion of report

Declaration

No portion of the work referred to in this report has been submitted in support of an application for another degree or qualification of this or any other university or other institute of learning.

Acknowledgements

I would like to express my sincere gratitude to my supervisor, Dr. Jonathan Shapiro, for his disproportionate support and guidance throughout this project. I also thank my second marker for taking their time to evaluate and assess my work. Moreover, Moreover, I am immensely grateful to my family and friends for their immeasurable support and for encouraging my ambitious pursuits throughout my degree.

Abbreviations & Acronyms

B | C | E | G | H | J | M | R | S | T | Y

B

BBC Black-Box Complexity. 13

BBO Black-Box Optimization. 13, 14, 21, 34, 37, 38, 44

BO Bayesian Optimization. 38

C

CI Continuous Integration. 51

CLI Command-Line Interface. 45, 46, 53

CNF Conjunctive Normal Form. 33

CR Cumulative Regret. 40, 56

E

EA Evolutionary Algorithms. 21, 26, 29, 30, 32, 35, 54, 55, 56, 59, 65, 67, 68, 69, 70, 71, 83

G

GA Genetic Algorithms. 32

H

HIFF Hierarchical If-and-Only-If. 32, 33, 64, 65, 68

J

JSON *acr. JavaScript Object Notation.* An open standard file format and data interchange format that uses human-readable text to store and transmit data objects consisting of name–value pairs and arrays (or other serializable values).. 5, 45, 46, 47, 48

M

MAB Multi-Armed Bandits. 38

MaxSAT Maximum Satisfiability problem. 33, 34

ML Machine Learning. 14

R

RLS Randomized Local Search. 29, 49, 63, 65, 66, 67, 68, 70

S

SA Simulated Annealing. 27, 28, 65

SAT Satisfiability problem. 33

SR Simple Regret. 41

T

TDD Test-Driven Development. 51

TTFO Time to First Optimum. 50, 51, 61, 65, 67, 69

Y

YAML *acr. YAML Ain't Markup Language.* A human-readable data serialization language, commonly used for configuration files.. 45, 46, 47, 48, 53

Chapter 1

Introduction

1.1 Motivation

Many aspects of our day-to-day lives revolves around finding optimality in problems that we can have no comprehensible insight about. It can be observed in industrial processes of ! TODO!. These types of problems are said to be Black-Box Optimization (BBO) problems.

We do not get much tools to interact with them save from a cost/objective function $\mathbf{f} : \mathcal{X} \mapsto \mathcal{S}$ (the 'oracle') that provides an abstract representation of the problem, where $\mathcal{X}$ is the search space, $\mathcal{S}$ the evaluation space !TODO: find concrete refs!. Typically, $\mathcal{S} \subseteq R$. The task is to minimize or maximize the solutions produced by this objective function, i.e., find a solution that Much study is done in optimizing this cost function without the liberty of classic mathematical tools such as continuity, smoothness and convexity (these methods warrant the oracle to return additional information, such as gradients, apart from the evaluation value of an input [46]).

1.1.1 Conventional Solutions

Classical analysis of solving BBO problems are concerned with the so-called Black-Box Complexity (BBC), that is, deriving the runtime bounds for an algorithm to sample a (global) optimum, depending only on the problem size $\mathbf{n}$. The reasoning behind this is palpable; the *computational* complexity of evaluating the objective function for such BBO systems are usually expensive.

A trivial example is Hyperparameter Tuning in Machine Learning (ML) models !CITE!. The optimal hyperparameters could be thought of as the *optima* for the Black-Box Optimization (BBO) problem of the tuning that maximizes their validation loss[1].

1.1.2 The Problem

While this approach secures the bounds on the first sampling of the optimum, it presents no clues on the solution quality produced by the algorithm. Consider the case when the algorithm samples the optimum at time t, then in the next step $t+1$, moves completely away from the vicinity. This algorithm might then produce worse answers, asymptotically.

1.1.3 Project's Alternative

[1] 'how well the model fits the underlying probability distribution of the data inputs & outputs'

1.2 Objectives

1.2.1 General Aims

1.2.2 Theoretical Exploration

1.2.3 Supplementary Software Development

1.2.4 Experimental Analysis

1.3 Project Outline

1.3.1 Literature Study

1.3.2 Isolating Relevant Benchmarks

1.3.3 Development of Experiment framework

1.3.4 Orchestrating Experiments

1.3.5 Evaluating Results

1.3.6

1.4 Ethical Considerations

1.5 Report Structure

1.5.1 Background

1.5.2 Software Design

1.5.3 Experiment Structure

1.5.4 Evaluation & Discussion of Results

1.5.5 Conclusion & Future Work

Chapter 2

Background

2.1 Computational Problems

2.1.1 Problem Formulations

A *computational problem* is a formal specification of a task defined over a set of *problem instances*. Each instance encodes the input data relevant to a particular occurrence of the problem. The *problem class* describes the collection of all such problem instances together with the corresponding output requirements. [32]

Problems are broadly categorized by the nature of their required output.

- A *decision problem* demands a binary answer, whether a solution exists for a problem instance.
- A *search problem* asks for the construction of an explicit solution satisfying a specified property, such as a feasible assignment or a path in a graph, or conclude no solution exists.
- An *optimization problem* further requires that the returned solution be optimal (or near-optimal) with respect to some given objective function.

The decision and search problem variants are closely related because algorithms solving the search variant can always be used to solve the decision variant. [32]

A further distinction concerns the role of randomness in the problem formulation. In a *deterministic* formulation, both the instance and every algorithm operating on it are fully specified without any probabilistic component. In a *stochastic* formulation, the instance itself, the objective evaluations, or the algorithm may

involve randomness. Noisy objective functions, where repeated evaluation of the same candidate solution may return different values, are a canonical example and arise naturally in simulation-based and real-world settings.

2.1.2 Algorithms

An *algorithm* is a finite, unambiguous sequence of instructions that transforms a given input into a desired output within a bounded number of steps. More precisely, an algorithm defines a computational procedure that, for each instance drawn from a specified problem class, terminates and produces a result satisfying the output requirements of that class [32]. Three properties are central to this definition: *finiteness*, in that the procedure halts after a finite number of steps; *definiteness*, in that each step is precisely specified with no ambiguity; and *effectiveness*, in that every operation is sufficiently basic to be carried out exactly. In the context of optimization, the input is a problem instance and the output is a candidate solution together with its objective value, with the algorithm's quality judged both by the solutions it returns and the computational resources it consumes in doing so.

2.1.2.1 Algorithm Properties

Deterministic and Randomized Algorithms. A *deterministic* algorithm produces the same sequence of operations and the same output on every execution for a given input. A *randomized* algorithm, by contrast, makes probabilistic choices during execution, so its behavior on a fixed input may vary across runs. In the black-box optimization setting, randomization is not merely a convenience but a necessity: the No-Free-Lunch theorems imply that no deterministic algorithm can perform well uniformly across all problem instances, and randomized search heuristics derive their practical value from their ability to escape local optima through stochastic perturbation [19, 32]. The theoretical analysis of such algorithms therefore concerns expected quantities (expected runtime, expected solution quality) rather than worst-case deterministic guarantees.

Adaptive vs. Non-Adaptive Algorithms An algorithm is *adaptive* if each action at step t is chosen as a function of the history of prior queries and their responses, $(x_1, f(x_1)), \ldots, (x_{t-1}, f(x_{t-1}))$; a *non-adaptive* algorithm such as pure

random sampling fixes all query points in advance and serves as a natural complexity baseline [22]. All algorithms studied in this project are fully adaptive. This sequential feedback structure is also the defining feature of online optimization frameworks such as the multi-armed bandit model (Section 2.3), where the learner's choice of arm at each round depends on all previously observed rewards. The regret-based performance measures developed in Section 2.4 are a direct consequence of this adaptive, sequential structure: they quantify the cumulative cost of not knowing the objective in advance, penalizing every step at which the algorithm has not yet identified the optimal region of the search space.

2.1.2.2 Oracle-Based Computation

An *oracle* is an abstract computational device that answers queries about a problem instance. In optimization, the most common oracle is the *function evaluation oracle*, which, given a query point $x \in \mathcal{X}$, returns the objective value $f(x)$ (or a noisy estimate thereof). The algorithm interacts with the oracle in a *query-response* model: it submits a query and receives a response before deciding on the next query. The total number of such interactions constitutes the *query complexity* of the algorithm, formalized in Section 2.1.5.2 [48, 22].

2.1.3 Search Space

2.1.3.1 Structure of the Search Space

The *search space* $\mathcal{X}$ is the set of all candidate solutions considered by an algorithm. Search spaces may be *finite* or *infinite*, and *discrete* or *continuous*.

Discrete combinatorial search spaces, in particular the Boolean hypercube $\{0,1\}^n$, are central to the theoretical analysis of evolutionary algorithms (EA) and other randomized search heuristics [21**?**] used in theoretical BBO analysis. Continuous spaces $\mathbb{R}^d$ arise in numerical optimization [48], and mixed spaces containing both discrete and continuous variables occur in practice but are considerably harder to analyze [6].

The structure of the search space, and in particular how neighboring solutions relate to one another, fundamentally influences the difficulty of optimization. This is encoded by a *neighborhood function* $\mathcal{N} : \mathcal{X} \to 2^{\mathcal{X}}$, assigning to each candidate solution the set of solutions reachable in a single step!CITE: Hoos and Stützle -

2005 - INTRODUCTION!. For the Boolean hypercube, the standard neighborhood consists of all strings at Hamming distance one !CITE: a resource using hamming distance!.

Remark 1 (Problem domain of interest)**.** *This project is concerned with* Pseudo-Boolean, Combinatorial Optimization problems*, with solution space* $\mathcal{X} \subseteq \{0,1\}^n$*. A candidate solution can be represented as a* bitstring*: a sequence of alphabets* $a \in \{0,1\}$*. The objective functions considered for such problems are of the form* $f : \mathcal{X} \to \mathbb{R}_0^+$*, where* $\mathbb{R}_0^+$ *denotes the non-negative reals. Without loss of generality, an objective function with co-domain* $\mathcal{T} \subseteq \mathbb{R}$ *can be mapped to this convention via the affine shift*

$$g : \mathbb{R} \to \mathbb{R}_0^+, \qquad g(x) = x - \min_{y \in \mathcal{X}} f(y),$$

which is well-defined whenever f *attains its minimum on* $\mathcal{X}$*, and preserves the ordering of all objective values by construction.*

2.1.3.2 Constraints and Feasibility

In a *constrained* optimization problem, the *feasible region* $\mathcal{F} \subseteq \mathcal{X}$ is defined by one or more constraints that a valid solution must satisfy [48, 9]. An *unconstrained* problem places no restriction beyond membership in $\mathcal{X}$. Constraints may be *hard* (a solution violating them is inadmissible) or *soft* (violations are penalized in the objective). The boundary of the feasible region can significantly complicate optimization, particularly for local search methods that may generate infeasible intermediate solutions.

2.1.4 Objective Functions and Landscape Properties

The geometric structure of the objective function determines both which algorithms are applicable and what convergence guarantees are attainable. A central organizing principle is that access to higher-order analytical information about f, i.e., gradients, curvature, or structural properties such as convexity, generally enables stronger theoretical guarantees and more efficient algorithms; restricting this access, as in the black-box setting, correspondingly limits what can be guaranteed.

2.1.4.1 Zeroth-, First-, and Second-Order Methods

Optimization algorithms can be classified by the order of analytical information they require from f. This *information hierarchy* has three tiers.

Zeroth-order methods use only evaluations of $f(x)$, with no gradient or curvature information. The algorithm observes only the output of the function evaluation oracle at each queried point and must infer search directions from comparisons of objective values alone. This exactly describes the black-box model formalized in Section 2.2: when the internal structure of f is inaccessible or analytically intractable, zeroth-order methods are the only option. Their weaker information access is the fundamental reason black-box optimization is harder in general than white-box optimization, and it is the regime in which all algorithms studied in this project operate.

First-order methods additionally access the gradient $\nabla f(x) \in \mathbb{R}^d$, which encodes the direction of steepest ascent at x. For a differentiable function $f : \mathbb{R}^d \to \mathbb{R}$, gradient-based methods exploit this to define descent directions and form the backbone of classical continuous optimization [48]. When f is convex but non-differentiable, the gradient generalizes to the *subdifferential* $\partial f(x)$, the set of all *subgradients* g satisfying

$$f(y) \geq f(x) + g^\top (y - x) \quad \forall y \in \mathbb{R}^d.$$

Subgradient methods replace the gradient with any element of $\partial f(x)$, recovering convergence under convexity alone, albeit at rates slower than those attainable with smooth gradients [9].

Second-order methods further access the Hessian $\nabla^2 f(x) \in \mathbb{R}^{d \times d}$, which captures local curvature. Methods that utilize or approximate the Hessian, such as Newton and quasi-Newton schemes, can achieve superlinear or quadratic convergence near a solution [48], at the cost of maintaining a $d \times d$ matrix.

Together, these three tiers define distinct levels of oracle access, and their availability directly governs the choice and efficiency of applicable algorithms. The move from second-order to zeroth-order corresponds to progressively weaker assumptions about what the oracle reveals, culminating in the black-box model where only function values are observable.

2.1.4.2 Landscape Properties

The following geometric properties of f characterize which tier of analytical information is both available and useful.

Convexity. A function $f : \mathbb{R}^d \to \mathbb{R}$ is *convex* if for all $x, y \in \mathbb{R}^d$ and $\lambda \in [0,1]$ [9],

$$f(\lambda x + (1-\lambda)y) \leq \lambda f(x) + (1-\lambda) f(y).$$

Convexity guarantees that every local minimum is global, rendering the landscape tractable for first-order methods. *Strong convexity* additionally requires

$$f(y) \geq f(x) + \nabla f(x)^\top (y-x) + \tfrac{\mu}{2}\|y-x\|^2$$

for some $\mu > 0$, implying a unique global minimum and linear convergence rates for gradient descent !CITE: Nesterov 2004!.

Smoothness and Lipschitz Continuity.

Ruggedness and Multimodality.

2.1.5 Complexity Measures

2.1.5.1 Classical Complexity: Time and Space

The classical measures of algorithmic efficiency are *time complexity* (the number of elementary operations performed as a function of input size) and *space complexity* (the amount of memory consumed). Both are typically expressed using asymptotic notation: $O(\cdot)$, $\Omega(\cdot)$, and $\Theta(\cdot)$ describe upper, lower, and tight bounds on resource usage as the input size n grows. In the setting of combinatorial optimization over $\{0,1\}^n$, the input size is the bitstring length n, and complexity results are stated as functions of n.

2.1.5.2 Information Complexity: Query and Oracle Complexity

In black-box optimization the internal structure of f is inaccessible, so the relevant measure of complexity is not the number of arithmetic operations but the number of oracle queries required to solve a problem. The *query complexity* of

an algorithm $\mathcal{A}$ on a problem class $\mathcal{F}$ is the number of calls to the function-evaluation oracle needed to identify a solution meeting the target criterion. The *black-box complexity* of a problem class is the minimum query complexity over all algorithms, formalizing the information-theoretic difficulty of the problem when only objective values are observable [22]. Lower bounds on black-box complexity establish that no algorithm can solve the problem in fewer queries, regardless of its computational power; upper bounds are witnessed by constructive algorithms. This duality mirrors the classical worst-case analysis of deterministic algorithms but is applied to the query-response model introduced in Section 2.1.2.2.

2.1.5.3 Expected vs. Worst-Case Analysis

For deterministic algorithms, worst-case analysis over all possible inputs is the standard framework. For randomized algorithms, and in particular for the randomized search heuristics studied in this project, the primary measure is the *expected runtime*: the mean number of function evaluations until a target solution is first found, averaged over the internal randomness of the algorithm [21, 19]. Expected runtime analysis is complemented by tail bounds (probabilistic guarantees that the runtime does not greatly exceed its expectation) derived via tools such as Markov's inequality and concentration arguments [19]. Worst-case analysis over problem instances, by contrast, motivates the minimax framework and Yao's minimax principle, which lower-bounds the expected complexity of any randomized algorithm by exhibiting a hard distribution over instances [22].

2.2 Black-Box Optimization

2.2.1 The Black-Box Model

In the *black-box optimization* model, the algorithm has no access to the analytical form of the objective function. It interacts with the objective exclusively through an oracle that, upon receiving a query $x \in \mathcal{X}$, returns $f(x)$ (or a noisy estimate thereof) [22, 19, 45]. This constitutes *zeroth-order* information: no gradient or higher-order information is available. The black-box model is appropriate whenever the objective arises from a simulation, physical experiment, or any process whose internal structure is inaccessible or too complex to exploit analytically.

In contrast, in *white-box optimization*, information about the problem instance is make available to the algorithm. This enables algorithms usage of some computational tools such as gradients, supporting optimization convergence.

2.2.1.1 Variants of Black-Box Models

2.2.2 Black-Box Complexity

2.2.3 Purpose of Randomized Search Heuristics

2.2.3.1 No-Free-Lunch Theorems

2.2.3.2 Taxonomy of Derivative-Free Methods

2.2.4 Randomized Search Heuristic Algorithms

2.2.4.1 Evolutionary Algorithms (EA)

Evolutionary algorithms are a broad family of population-based, randomized search heuristics inspired by biological evolution. In discrete optimization, the canonical paradigm maintains one or more *candidate solutions* (bit strings) and iterates two operators: *mutation*, which perturbs a solution stochastically, and *selection*, which retains solutions according to their objective value [21, 19]. The simplest instance of this paradigm, the $(1+1)$ EA, serves as the theoretical baseline throughout this project.

The $(1+1)$ EA. The $(1+1)$ EA maintains a single current solution $x \in \{0,1\}^n$. In each iteration it produces one offspring y by *standard bit mutation*: every bit

is flipped independently with probability p. The offspring replaces the parent if and only if $f(y) \geq f(x)$ (*elitist selection*). With the standard choice $p = 1/n$ the expected number of flipped bits is exactly one. The resulting process is a homogeneous Markov chain on $\{0,1\}^n$ with a non-decreasing fitness sequence [21, 61].

Algorithm 1 The $(1+1)$ EA with mutation probability $p \in (0,1)$

```
Sample x ∈ {0,1}^n uniformly at random; evaluate f(x)
repeat
    y ← x
    for i = 1, ..., n do
        flip y_i with probability p, independently
    end for
    if f(y) ≥ f(x) then x ← y
    end if
until termination criterion met
```

The classical runtime results for $p = 1/n$ are

$$\mathbb{E}[T_{\textsc{OneMax}}] = e\,n \ln n + O(n) \qquad \text{and} \qquad \mathbb{E}[T_{\textsc{LeadingOnes}}] = \tfrac{e-1}{2}\,n^2 + o(n^2),$$

and the $\Theta(n \log n)$ bound extends to *all* linear pseudo-Boolean functions with non-zero weights [61]. On general landscapes the algorithm may require exponential time: even simple quadratic and unimodal functions can trap it indefinitely without restarts or adaptive mutation [21]. Advanced variants and implementation-aware refinements are discussed in Appendix A.

2.2.4.2 Simulated Annealing

Simulated Annealing (SA) is a probabilistic local search algorithm that extends iterative improvement by occasionally accepting worsening moves, allowing escape from local optima that would otherwise trap a pure hill-climber. The method was introduced by Kirkpatrick et al. [41], who drew an explicit analogy with the physical process of annealing in solids: a material is first melted at high temperature, then cooled slowly so that atoms settle into a low-energy crystalline configuration rather than a disordered glassy state.

In the optimization context, the algorithm maintains a current solution $x \in \mathcal{X}$ and at each step proposes a random neighbor $y \in \mathcal{N}(x)$. If $f(y) \geq f(x)$, the move is accepted unconditionally. If $f(y) < f(x)$, the move is accepted with probability

$$\exp\left(\frac{f(y) - f(x)}{T(t)}\right),$$

where $T(t) > 0$ is the *temperature* at step t, a control parameter that decreases over time according to a *cooling schedule* [41, 10]. At high temperatures the acceptance criterion is nearly uniform, permitting broad exploration; as $T(t) \to 0$ the algorithm collapses toward a greedy hill-climber that accepts only improvements.

The theoretical behavior of SA is governed by the cooling schedule. Modelling the process as a time-inhomogeneous Markov chain, Hajek [29] established the sharp necessary and sufficient condition for convergence in probability to the global optimum: the schedule must satisfy

$$\sum_{t=1}^{\infty} \exp\left(\frac{-d^*}{T(t)}\right) = +\infty,$$

where d^* is the depth of the deepest local minimum that is not globally optimal. For the logarithmic schedule $T(t) = d/\log(t+1)$, this condition reduces to the requirement $d \geq d^*$: the scaling constant must be at least as large as the hardest energy barrier in the landscape [29, 10]. In practice, logarithmic cooling is theoretically optimal but often too slow; exponential and linear schedules cool faster at the cost of losing the convergence guarantee, reducing SA to a stochastic hill-climber once the temperature reaches zero. The choice of cooling schedule therefore encodes a fundamental trade-off between solution quality and computational budget, and is the primary design decision when applying SA to any combinatorial problem [41, 29].

Cooling Schedules. Three schedule families are studied in this project and are implemented in the experimental framework. *Logarithmic cooling*, $T(t) = d/\log(t+1)$, maintains a strictly positive temperature for all finite t and satisfies Hajék's convergence condition when $d \geq d^*$ [29]. *Exponential cooling*, $T(t) = T_0 \cdot \alpha^t$ for $\alpha \in (0,1)$, is the schedule used in the original simulated annealing experiments [41]; it decays geometrically and effectively reaches zero within the evaluation budget, after which SA reduces to a deterministic hill-climber. *Linear*

cooling, $T(t) = \max(T_f, T_0 - (T_0 - T_f)\, t/t_{\max})$, interpolates uniformly between an initial and final temperature over a fixed horizon; like the exponential schedule, it carries no convergence guarantee once the temperature floor T_f is reached [10]. The regret implications of each schedule, and in particular the point at which temperature collapse occurs, and instantaneous regret stabilizes, are examined empirically in Suite 11.

Cooling Schedules

2.2.4.3 Randomized Local Search (RLS)

Stochastic Hill-Climber - Greedy RLS

$\epsilon - greedy$ **RLS**

Decaying $\epsilon - greedy$ **RLS**

2.2.5 Few Pseudo-Boolean Problems

The *pseudo-Boolean* setting considers functions $f : \{0,1\}^n \to \mathbb{R}$ defined on binary strings of length n. Several benchmark functions are widely used to study and compare the behavior of randomized search heuristics, each encoding a different type of difficulty. [45, 22]

2.2.5.1 OneMax

The OneMax function counts the number of one-bits in a bit string:

$$\textsc{OneMax}(x) = \sum_{i=1}^{n} x_i, \qquad x \in \{0,1\}^n.$$

The unique global maximum is $x^* = 1^n$ with $\textsc{OneMax}(1^n) = n$. OneMax is the canonical baseline benchmark: the landscape is unimodal and every improving move increases the function value by exactly one, providing a clear gradient for local search methods. The expected runtime of the $(1+1)$ Evolutionary Algorithms (EA) on OneMax is $\Theta(n \log n)$. !CITE: Droste, Jansen, Wegener (2002)!

2.2.5.2 LeadingOnes

The LeadingOnes function counts the length of the maximal leading prefix of ones:

$$\textsc{LeadingOnes}(x) = \sum_{i=1}^{n} \prod_{j=1}^{i} x_j.$$

Equivalently, $\textsc{LeadingOnes}(x)$ equals the index of the first zero bit minus one, or n if no zero exists. The unique global optimum is again 1^n. Because progress requires the leading bit to be set before any subsequent bit can contribute, LeadingOnes is significantly harder than OneMax: the expected runtime of the $(1+1)$ EA is $\Theta(n^2)$. !CITE: Droste, Jansen, Wegener (2002)!

2.2.5.3 Jump

The Jump function with parameter $k \in \{1, \ldots, n\}$ coincides with OneMax on bit strings with at most $n-k$ ones and on the global optimum 1^n, but introduces a region of reduced fitness for strings with between $n-k+1$ and $n-1$ ones. This creates a "gap" of width k that algorithms relying on single-bit improvements cannot cross: the set of strings with exactly $n-k$ ones constitutes a local optimum of standard local search, and reaching the global optimum 1^n requires flipping all k remaining zero bits simultaneously. For the $(1+1)$ EA with standard bit mutation at rate $1/n$, the probability of a specific k-bit flip is $\Theta(n^{-k})$, giving an expected runtime of $\Theta(n^k)$ on $\textsc{Jump}_k$.

Formally, $\textsc{Jump}_{k,n} \colon \{0,1\}^n \to \mathbb{R}$ is defined as

$$\textsc{Jump}_{k,n}(x) \;=\; \begin{cases} k + \sum_{i=1}^{n} x_i & \text{if } \sum_{i=1}^{n} x_i \le n-k \text{ or } \sum_{i=1}^{n} x_i = n, \\ n - \sum_{i=1}^{n} x_i & \text{otherwise.} \end{cases}$$

Droste, Jansen, and Wegener [21] proved that the expected running time of the $(1+1)$ EA on $\textsc{Jump}_{k,n}$ is $\Theta(n^k + n \log n)$ for $k \in \{1, 2, \ldots, n\}$, which simplifies to $\Theta(n^k)$ for $k \ge 2$.

2.2.5.4 BinaryValue[BinVal]

The BinaryValue (BinVal) function interprets a bit string as a binary integer by weighting bits exponentially by position:

$$\textsc{BinVal}(x) = \sum_{i=1}^{n} 2^{i-1} x_i.$$

The global optimum is 1^n with value $2^n - 1$. The exponential weighting means that the most significant bit contributes more than the combined value of all less significant bits, making BinVal highly structured.

2.2.5.5 Trap

The Trap function is a piecewise-linear *function of unitation* formally analyzed for deception conditions by Deb and Goldberg [18]. Its value depends only on the number of ones $u = \sum_{i=1}^{n} x_i$, not on their positions, and with slope-change location $z = n - k$ it is defined as

$$\textsc{Trap}_k(x) = \begin{cases} \frac{n}{z}(z-u) & \text{if } u \le z, \\ \frac{n}{k}(u-z) & \text{otherwise.} \end{cases}$$

For $u \le z$ the gradient points toward the all-zeros attractor 0^n; only past the slope change does fitness increase toward the global optimum 1^n. Every improving move for a hill-climber therefore moves *away* from the optimum. Deb and Goldberg establish that the necessary and sufficient condition for full deception is that all schemata of order $n-1$ favor the local optimum, which requires $z \ge \lfloor n/2 \rfloor$ [18].

2.2.5.6 Plateau

The Plateau function with parameter k is defined as OneMax everywhere except in a ball of radius k around the optimum, where fitness is held constant at the second-best value $n - k$ [2]:

$$\textsc{Plateau}_k(x) = \begin{cases} n-k & \text{if } n-k < \textsc{OneMax}(x) < n, \\ \textsc{OneMax}(x) & \text{otherwise.} \end{cases}$$

The global optimum is 1^n with value n. Unlike JUMP, which penalizes solutions near the optimum with a *reduced* fitness, PLATEAU assigns the plateau region the same fitness as the boundary level $n-k$, so there are no true local optima. Instead, the algorithm performs an unguided random walk across the $\Theta(n^k)$ plateau points, gaining no fitness signal until it reaches 1^n by flipping between one and k zero-bits simultaneously.

Antipov and Doerr [2] establish that for any unbiased mutation operator the expected runtime of the $(1+1)$ EA is

$$\mathbb{E}[T] = \frac{n^k}{k!\,\Pr[1 \le \alpha \le k]}(1+o(1)),$$

where α is the number of bits flipped per step. This depends only on the probability to flip between one and k bits, not on other properties of the operator. The optimal mutation rate under standard bit mutation is approximately $k/(en)$, a factor of $\Theta(k)$ above the canonical $1/n$ recommendation.

2.2.5.7 Hierarchical If-and-Only-If (HIFF)

The HIERARCHICAL IF-AND-ONLY-IF (HIFF) function is a benchmark designed to model *hierarchical building-block interdependency*: a property absent from earlier Genetic Algorithms (GA) test problems, which treat sub-problems as separable [59]. In HIFF, the optimal solution to a block of bits depends on how that block combines with its neighbors at the next level of the hierarchy, making sub-problems *non-separable* in a consistent, multi-level sense.

The fitness of a bitstring $x \in \{0,1\}^n$, where n must be a power of two, is defined recursively. A block B of length one always contributes a fitness of one. For any longer block, if all bits in B are identical (all zeros or all ones), the block contributes $|B|$ to fitness and its two halves are recursively evaluated; otherwise only the recursive evaluation of the two halves contributes. Formally,

$$f(B) = \begin{cases} 1 & \text{if } |B| = 1, \\ |B| + f(B_L) + f(B_R) & \text{if } |B| > 1 \text{ and all bits in } B \text{ are equal}, \\ f(B_L) + f(B_R) & \text{otherwise}, \end{cases}$$

where B_L and B_R denote the left and right halves of B. The global optimum is achieved by the all-zeros or all-ones bitstring, giving a maximum fitness of

$n(\log_2 n + 1)$. Two global optima exist and are maximally distant from one another, a structural feature that makes HIFF acutely challenging for hill-climbing algorithms: a single bit-flip from either optimum disrupts the top-level block uniformity and causes a sharp fitness drop, so local search stagnates at intermediate solutions in which only lower-level blocks are uniform [59].

The landscape is highly multimodal. Secondary optima consist of strings with contiguous blocks of zeros and ones at each hierarchical level (for example, the first $n/2$ bits all zero and the remaining $n/2$ all one for a length-n string), and these local optima are maximally distant from both global optima. Oates et al. [49] demonstrate empirically that this structure induces a characteristic tri-modal performance profile in evolutionary search: progress occurs in discrete phases corresponding to the successive exploitation of one-bit, two-bit, and higher-order mutations needed to escape each tier of local optima. This property makes HIFF a useful diagnostic benchmark for regret analysis, as the hierarchical barrier structure should produce a distinctive staircase signature in the cumulative regret trajectory.

2.2.5.8 MaxSAT

Maximum Satisfiability problem (MaxSAT) is the optimization variant of the Boolean Satisfiability problem (SAT). A propositional formula in Conjunctive Normal Form (CNF) is a conjunction of *clauses*, each of which is a disjunction of *literals* (a Boolean variable or its negation). Given such a formula with m clauses over n Boolean variables, the goal is to find a truth assignment that maximizes the number of satisfied clauses. MaxSAT is NP-hard in general [33] and serves as an important benchmark for combinatorial optimization algorithms.

Petersen Graph 3-Coloring. The *Petersen graph* [60] is a well-known 3-regular graph on ten vertices and fifteen edges whose chromatic number is exactly three, meaning a valid 3-colouring exists and no 2-colouring does. This project encodes the 3-colouring problem as a MaxSAT instance over $n = 30$ binary variables, with one variable per vertex-color pair: variable $x_{v,c}$ is 1 if vertex v is assigned color $c \in \{0, 1, 2\}$. Three families of soft clauses encode the coloring constraints: ten *at-least-one* clauses (each vertex must receive at least one color), thirty *at-most-one* clauses (each vertex must not hold two colors simultaneously), and forty-five *edge-conflict* clauses (adjacent vertices must not share a

color), giving $m = 85$ clauses in total. Because the Petersen graph is 3-colourable, all 85 clauses can be satisfied simultaneously, so the global optimum is $f^* = 85$ and is genuinely achievable [33]. This is the key advantage over random MaxSAT instances, whose declared optima are rarely reachable in practice. The instance therefore provides a controlled combinatorial benchmark with a known, verifiable optimum, complementing the pseudo-Boolean problems studied in this project.

2.2.5.9 Tunable Landscapes: NK-Landscapes

NK-landscapes [40] are a parametric family of pseudo-Boolean functions controlled by two integers: N (the number of variables) and K (the number of epistatic interactions per variable, $0 \leq K \leq N-1$). The fitness of a string $x \in \{0,1\}^N$ is defined as

$$f(x) = \frac{1}{N} \sum_{i=1}^{N} f_i\Big(x_i, x_{\pi(i,1)}, \ldots, x_{\pi(i,K)}\Big),$$

where each local contribution $f_i : \{0,1\}^{K+1} \to [0,1]$ is drawn independently and uniformly at random, and $\pi(i,\cdot)$ denotes the K epistatic neighbors of variable i.

When $K = 0$ the contributions are independent, producing a smooth, additively separable landscape analogous to OneMax. As K increases toward $N-1$, epistatic interactions intensify and the landscape becomes increasingly rugged, with the number of local optima growing in the limit. NK-landscapes thus provide a tunable model of ruggedness and are widely used to study algorithmic scalability and robustness across a spectrum of problem difficulties.

2.2.6 Traditional Performance Measures in BBO

The theoretical analysis of BBO algorithms has developed several complementary frameworks for quantifying performance. Each framework fixes one dimension of the problem, either the evaluation budget or the target solution quality, and asks what can be guaranteed along the other. Together they form the standard vocabulary for comparing randomized search heuristics, yet all share a common limitation that motivates the regret-based view developed in Section 2.4.

2.2.6.1 Expected Runtime (Hitting Time)

The predominant performance measure in the runtime analysis of evolutionary algorithms is the *expected optimization time*: the expected number of function evaluations until an optimal solution is queried for the first time [21, 19]. Formally, for a black-box algorithm producing query sequence $x_1, x_2, \ldots$, the *first hitting time* of the global optimum is

$$\tau_{f^*} := \min\Big\{t \in \mathbb{N} : f(x_t) = f^*\Big\},$$

and the expected runtime is $\mathbb{E}[\tau_{f^*}]$. This quantity has proved immensely useful for establishing complexity hierarchies and ranking algorithms; the results $\Theta(n \log n)$ for the $(1+1)$ EA on OneMax, $\Theta(n^2)$ on LeadingOnes, and $\Theta(n^k)$ on Jump_k are canonical examples [21]. The primary analytical tools for deriving such bounds are drift theory [45], the fitness-level method, and Markov chain arguments [19].

2.2.6.2 Fixed-Budget Analysis

In *fixed-budget analysis* [38], the evaluation budget T is fixed in advance and the quantity of interest is the expected quality of the best solution found within that budget. Denoting by $f_T^*(\mathcal{A})$ the best objective value achieved by algorithm $\mathcal{A}$ within T evaluations, the fixed-budget framework characterizes $\mathbb{E}[f_T^*(\mathcal{A})]$ as a function of T [38]. This perspective is practically motivated: in almost all realistic settings a user cannot detect when the global optimum has been found, and the algorithm must simply return the best solution found when the budget is exhausted [52]. As a consequence, comparing two algorithms under a fixed budget may reverse the conclusions drawn from comparing their expected runtimes: an algorithm with a larger $\mathbb{E}[\tau_{f^*}]$ may still produce superior intermediate solutions within a given budget [38]. This framework connects directly to the simple regret defined in Section 2.4.2.3: maximizing $\mathbb{E}[f_T^*(\mathcal{A})]$ is equivalent to minimizing expected simple regret at budget T.

2.2.6.3 Fixed-Target Analysis and Runtime Profiles

Fixed-target analysis generalizes the expected runtime by considering not only the global optimum but every intermediate fitness level. For a target quality v,

the *first hitting time* is

$$\tau_v := \min\Big\{t \in \mathbb{N} : f(x_t) \geq v\Big\},$$

and the goal is to characterize the distribution of τ_v for all relevant v. This viewpoint appears in the analysis of restricted black-box models; for example, Doerr and Lengler [20] derive hitting-time bounds across all fitness levels for OneMax under several information-theoretic restrictions, revealing how model constraints shift the difficulty at each tier of the landscape.

Pinto and Doerr [52] formalize this into the *runtime profile* framework: the *expectation-based runtime profile* $\Big(\mathbb{E}[\tau_v]\Big)_v$ and the strictly finer *distributional runtime profile* $\mathcal{P}(v,t) := \mathbb{P}(\tau_v > t)$ together capture the full anytime behavior of an algorithm rather than only its behavior at the endpoint. Crucially, runtime profiles can identify *crossover fitness levels* where one algorithm overtakes another in the quality trajectory, behavior invisible to the single expected optimization time. The formal development, including the connection between distributional runtime profiles and cumulative regret, is given in Section 4.3.

2.2.6.4 Limitations and the Case for a Continuous Performance Measure

Despite their individual strengths, all three frameworks above share a structural limitation: they measure algorithm performance at isolated thresholds rather than over the entire quality trajectory. The expected runtime reduces the run to a single hitting event; fixed-budget analysis reduces it to a single scalar per budget value; and even the runtime profile, while richer, is a collection of per-level summaries rather than a unified measure of trajectory quality. Crucially, none of these measures penalizes an algorithm for spending many evaluations at low fitness values: a strategy that stagnates near the starting fitness and then discovers the optimum in a single lucky step is indistinguishable from one that makes steady, monotone progress, as far as $\mathbb{E}[\tau_{f^*}]$ is concerned.

Pinto and Doerr [52] identify this gap explicitly, calling for *combined measures* capable of describing the anytime behavior of an evolutionary algorithm within a unified framework, and naming regret-based measures from machine learning as a natural candidate, while noting that a fully satisfying formulation remains an open problem. This project takes that call as its starting point. The sequential

decision-making framework and the formal vocabulary of regret are developed in Sections 2.3–2.4; their synthesis with BBO performance analysis is the subject of Section 2.5.

2.3 Sequential and Online Optimization Frameworks

2.3.1 Exploration-Exploitation Trade-off

Sequential black-box optimization is governed by the exploration–exploitation trade-off. At each round, the algorithm must either explore uncertain regions of the search space to gain information or exploit the currently best-known region to obtain good values immediately [15, 12, 44] Since each evaluation is both a measurement and a search step, neither extreme is satisfactory: pure exploitation can lock onto a suboptimal region, while pure exploration wastes budget.

This tension is central to regret-based analysis. Regret measures the cost of not knowing the objective in advance, so it captures both slow identification of good regions and repeated evaluation of poor ones.

2.3.2 Multi-Armed Bandits

The MAB model is the basic framework for sequential decision-making under partial feedback [12, 44]. In the stochastic K-armed setting, an algorithm chooses an arm $A_t \in \{1, \ldots, K\}$ at each round and observes only the reward of that arm. If arm i has mean reward μ_i and $\mu^* = \max_i \mu_i$, the standard pseudo-regret is

$$\bar{R}_T = T\mu^* - \sum_{t=1}^{T} \mu_{A_t} = \sum_{i:\Delta_i>0} \Delta_i \, \mathbb{E}[N_i(T)],$$

where $\Delta_i = \mu^* - \mu_i$ is the sub-optimality gap and $N_i(T)$ is the number of pulls of arm i up to time T [12, 44]. Regret is therefore driven by how often the learner samples suboptimal actions.

This model connects naturally to BBO. In a finite search space, candidate solutions can be viewed as arms and objective evaluations as arm pulls. For large or structured domains, however, this flat view breaks down, motivating more structured models such as Bayesian optimization.

2.3.3 Bayesian Optimization

Bayesian Optimization (BO) treats expensive black-box optimization as a sequential learning problem. The unknown objective is modelled by a probabilistic

surrogate, typically a Gaussian process, and an acquisition function selects the next query by balancing predicted value against uncertainty [39, 55, 24].

For this chapter, Bayesian optimization matters mainly because it is the clearest bridge between regret theory and black-box optimization. In particular, GP-UCB admits sublinear cumulative-regret guarantees under Gaussian-process or RKHS[1] assumptions [57, 16, 54]. At the same time, these guarantees rely on surrogate assumptions that are much more natural in low-dimensional continuous spaces than in pseudo-Boolean domains such as $\{0,1\}^n$. These makes regret an established concept in black-box optimization, but not yet one that transfers directly to the combinatorial heuristics studied here.

[1] *abbr.* Reproducing kernel Hilbert space. (Optionally see [25])

2.4 Computational Regret

2.4.1 Definition of Regret

Let $f : \mathcal{X} \to \mathbb{R}$ be an objective function with global optimum value $f^* = \max_{x \in \mathcal{X}} f(x)$. An algorithm that queries points $x_1, x_2, \ldots, x_T$ incurs *regret* by querying suboptimal solutions. Regret quantifies the gap between the algorithm's performance and that of an oracle always querying the optimum [15].

2.4.2 Types of Regret

2.4.2.1 Instantaneous Regret

The *instantaneous regret* at round t is

$$\rho_t = f^* - f(x_t),$$

capturing the suboptimality of the single query made at step t. A related variant tracks the *incumbent* (best-so-far) solution rather than the current query:

$$\rho_t^{\text{inc}} = f^* - f(\hat{x}_t), \quad \hat{x}_t = \arg\max_{s \le t} f(x_s).$$

Because $\hat{x}_t$ is non-decreasing in quality, ρ_t^{inc} is monotonically non-increasing over t, and is equivalent to simple regret evaluated at time t(2.4.2.3).

2.4.2.2 Cumulative Regret

Two cumulative regret variants arise naturally and serve different purposes.

The *Cumulative Regret (CR)* after T rounds is [15]

$$R_T = \sum_{t=1}^{T} \rho_t = \sum_{t=1}^{T} \Big(f^* - f(x_t) \Big).$$

This is the standard formulation from the online learning literature: it sums the instantaneous sub-optimality of each individual query, regardless of what the algorithm has previously observed. Minimizing R_T corresponds to performing well *throughout* the optimization process, penalizing every suboptimal query equally.

The *incumbent cumulative regret* instead tracks the best-so-far process

$$B(t) := \max_{s \le t} f(x_s),$$

yielding

$$R_T^{\text{inc}} = \sum_{t=1}^{T} \rho_t^{\text{inc}} = \sum_{t=1}^{T} \Big(f^* - B(t) \Big).$$

Since $B(t)$ is non-decreasing in t, the per-step contributions ρ_t^{inc} are non-increasing, and $R_T^{\text{inc}} \le R_T$ for all T. The incumbent formulation is the appropriate notion when the practitioner retains the best solution found rather than only the most recent query, which is the standard convention for the randomized search heuristics studied in this project. All regret analyses in Chapter 5 use R_T^{inc} unless otherwise stated.

In continuous time, the incumbent cumulative regret is approximated via a left-hold (piecewise-constant) integral

$$R_T^{\text{inc}} \approx \sum_{t=1}^{T} \rho_{t-1}^{\text{inc}} \, \Delta t_t, \quad \Delta t_t = t - t_{t-1},$$

which accounts for unequal spacing between evaluations when function evaluation time is non-uniform. In all experiments reported here, evaluations are unit-cost and time is measured in discrete query counts, so $\Delta t_t = 1$ throughout and this reduces to the standard discrete sum.

2.4.2.3 Simple Regret

The *Simple Regret (SR)* after T rounds is [13]

$$r_T = f^* - f(\hat{x}_T),$$

where $\hat{x}_T = \arg\max_{t \le T} f(x_t)$ is the best solution encountered. Simple regret measures the quality of the *final output* and is the appropriate notion when the goal is to return a good solution rather than to perform well during the search. Note that $r_T = \rho_T^{\text{inc}}$: simple regret is the incumbent instantaneous regret at the final round.

When comparing algorithms across multiple independent runs, the standard

scalar summary is the *expected simple regret*

$$\mathbb{E}[r_T] = \frac{1}{N}\sum_{i=1}^{N} r_T^{(i)},$$

where $r_T^{(i)}$ denotes the simple regret of the i-th run.

2.4.2.4 Normalized Regret

To enable comparison across problem instances with different objective scales, simple regret can be normalized by the range of attainable fitness values:

$$\tilde{r}_T = \frac{f^* - f(\hat{x}_T)}{f^* - f_{\text{worst}}} \in [0, 1],$$

where f_{worst} is the worst observed or theoretical fitness value. A value of 0 indicates an optimal solution was found; a value of 1 indicates no improvement over the worst case. When $f^* = f_{\text{worst}}$ (a degenerate constant landscape), the problem is trivially solved and normalized regret is defined as 0.

2.4.3 Regret Bounds

Regret analysis studies how regret grows with the budget T. The central benchmark is sublinear regret: if $R_T = o(T)$, then the average regret R_T/T tends to zero, meaning that the algorithm asymptotically performs as well as an oracle with full information [15, 12, 44].

In finite stochastic bandits, classic upper bounds show that algorithms such as UCB1 achieve logarithmic gap-dependent regret and sublinear worst-case regret [7]. Matching lower-bound results show that logarithmic regret is, in general, unavoidable up to constants under standard assumptions [43]. In Gaussian-process bandits, both upper and lower bounds depend on information gain and kernel structure rather than only on the number of actions [57, 16, 54]. These results are important here mainly as a template: regret can be treated as a complexity measure for sequential optimization.

2.4.4 Connections to Runtime Analysis

2.5 Regret-Aware Black-Box Optimization

Regret provides a fine-grained complexity measure in the black-box setting: rather than simply asking whether an algorithm finds the optimum within T queries, regret tracks how suboptimal the queries are throughout the process. This is particularly relevant when every evaluation is costly and performing well before the budget is exhausted matters in practice.

2.5.1 Regret as a Performance Measure for Randomized Search Heuristics

For randomized search heuristics, regret complements rather than replaces runtime analysis. Runtime asks when a target is first reached; regret asks how good the algorithm's queried or retained solutions are throughout the run. Fixed-budget analysis already moves in this direction, since expected simple regret is just the expected final sub-optimality after a budget of T evaluations [38, 13].

The distinction matters operationally. Low simple regret is appropriate when only the final returned solution matters. Low cumulative regret is appropriate in anytime settings, where useful intermediate performance also matters [15, 52]. For the heuristics considered in this project, the incumbent-based formulation is the natural one because the algorithm's output at interruption is the best-so-far solution.

2.5.2 Related Work: Regret in Bayesian and Surrogate-Based Optimization

2.5.3 Related Work: Regret-Inspired Analysis of Evolutionary Algorithms

2.5.4 Related Work: Bandit-Inspired Metaheuristics

2.5.5 Connections to Runtime Analysis via Drift Theory

2.5.6 Open Questions and Project Scope

Chapter 3

Supplementary Software Design

This chapter discusses the supplementary software developed for carrying out multiple experiments on *simulated* BBO problems efficiently. Trivially, analysis cannot be carried out on truly black-box instances; the problems are emulated to be black-box by providing only the *zeroth-order* information.

3.1 Purpose of the Software

A supplementary experimental framework, `regret`, was developed to support the orchestration and analysis of experiments on pseudo-Boolean and combinatorial optimization problems. The framework was motivated by the absence of readily available tooling that combines (i) flexible, declarative batch experiment execution, (ii) trajectory-level data collection, and (iii) regret-specific analysis pipelines in a single cohesive system.

Existing platforms such as Google Vizier [26] and EvoX [34] address hyperparameter optimization and population-based search respectively, but neither provides native support for regret metrics, runtime profile analysis, or the configuration-first experiment execution model required for this project's experimental protocol. `regret` was therefore purpose-built to fill this gap, providing a reproducible, extensible environment for studying simple, instantaneous, and cumulative regret across a controlled suite of algorithms and benchmark problems.

3.1.1 Preexisting software

Several established frameworks were considered prior to development. **Google Vizier** [26] provides a scalable black-box optimization service with a client–server architecture, but is oriented toward hyperparameter tuning rather than controlled algorithmic benchmarking, and does not expose trajectory-level data in a form amenable to regret analysis. **EvoX** [34] is a JAX-based evolutionary computation library offering GPU-accelerated population methods, but its abstraction layer couples algorithm state to framework internals in ways that preclude the lightweight callback-driven trajectory recording required here. General-purpose scientific workflow systems such as Snakemake [47] and Luigi [23] offer pipeline orchestration but impose significant configuration overhead for the combinatorial (algorithm × problem × budget) sweeps central to this study.

3.1.2 Capabilities

The framework supports the full experimental life-cycle through a single Command-Line Interface (CLI), `run_experiment`, which accepts one or more YAML configuration files and executes the appropriate sub-command. Table 3.1 summarizes the available commands.

Table 3.1: CLI sub-commands exposed by the `run_experiment` entry point (equivalently `python -m regret`).

Command	Description
`validate`	Parse and validate one or more YAML configurations against the JSON Schema and semantic registry checks without executing any computation.
`plan`	Print a dry-run summary: problem/algorithm combinations, total runs, and evaluation budgets.
`run [--no-plot]`	Execute one or more experiment configurations; optionally suppress plot generation after completion.
`analyze`	Regenerate all plots from previously saved JSON results without re-running experiments.
`table [--format]`	Generate summary tables from existing results in `latex`, `csv`, or `markdown` format.

Batch Experiment Execution. All experiments are specified declaratively via YAML configuration files validated against a JSON Schema prior to execution.

A configuration defines the suite name, evaluation budgets, run count, output mode, and lists of problem and algorithm specifications. Problems and algorithms are referenced by string keys resolved against internal registries, providing a clean separation between configuration and implementation.

Parallelised execution. When `parallel: true` is set in the YAML configuration, independent runs are initiated via Python's `ProcessPoolExecutor`, with per-run random seeds fixed to indices $0, 1, \ldots, r-1$ to ensure full reproducibility. Results are reordered by seed index after completion.

Flexible output modes. In `lite` mode, only summary statistics are persisted per run. In `full` mode, the complete evaluation trajectory (a sequence of `(evaluation_index, current_value, best_value)` tuples) is recorded and stored alongside aggregate statistics in structured JSON files.

Batched multi-configuration execution. All sub-commands accept multiple configuration files, enabling sequential execution of logically grouped experiment suites from a single invocation.

3.1.3 Design Decisions

The software was intended to be extensive & configurable for other developers and users to interact with quickly, with options for modularity when required. This was enabled by the adoption of configuration-first, job dispatch policy for independent and declarative experimentation. Problems and Algorithms to test against can easily be defined by extending abstract classes `Problem` & `Algorithm`, ensuring a common interface for consistent interactions.

3.1.3.1 Independent Job Execution framework

The CLI is implemented using **Typer** [53], which maps sub-commands to typed Python functions with automatic argument parsing. The "orchestration" layer coordinates problem & algorithm instantiation, result persistence, and plot generation, delegating individual runs to `ExperimentRunner`. Parallel execution uses `ProcessPoolExecutor` from Python's standard `concurrent.futures` module; results are collected via `as_completed` and reordered by seed for deterministic

output ordering. Figure 3.1 illustrates the full validation and the experiment (job) dispatch pipeline.

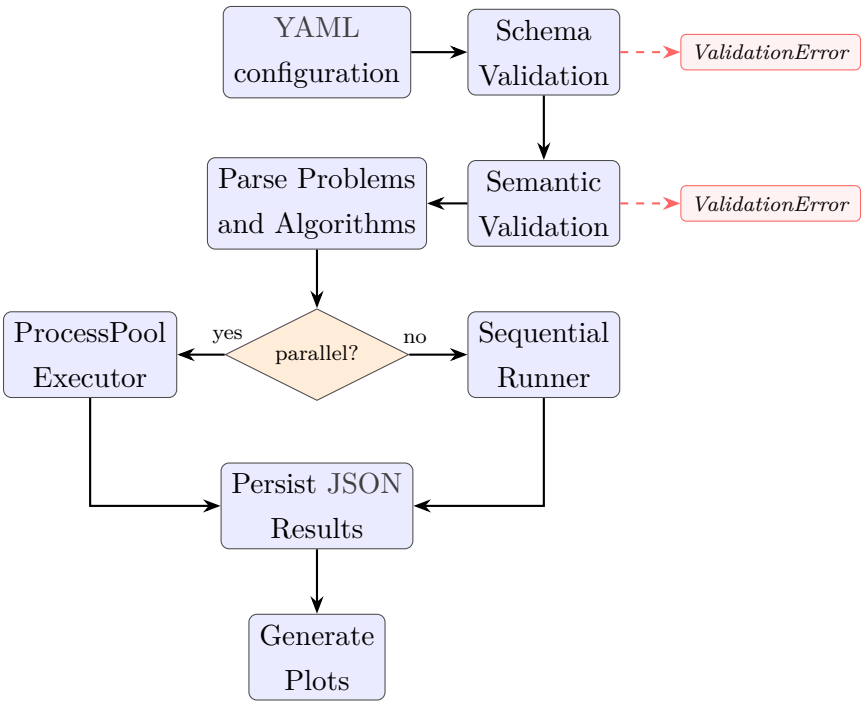


Figure 3.1: Validation and job dispatch pipeline. YAML configuration files are validated structurally via JSON Schema and then semantically against class registries before execution begins. Any validation failure raises a `ValidationError` and halts the pipeline before computation begins.

3.1.3.2 Raw Output JSON files

The output JSON files are *compacted* by replacing redundant stylistic whitespaces and newlines with other seperators. This reduced the raw file sizes by upto 50%. JSON files were preferred for storing the raw files since it included *mixed* data, including metadata, general statistics for each run, along with the trajectories. A 'NoSQL' (key-value) database was considered but not used since the experiments were relatively small scale.

3.1.3.3 UML Diagram

Figure 3.2 presents the core class structure of the framework. Abstract base classes `Problem` and `Algorithm` define the shared interface enforced across all concrete implementations. Concrete subclasses are registered by string key and

resolved at runtime. `ExperimentRunner` orchestrates individual runs and delegates result persistence, while the `CoolingSchedule` protocol is resolved from YAML at parse time via a *registry*.

3.1.4 Limitations

The framework is currently constrained to **binary search spaces** $\{0,1\}^n$; continuous and integer-valued domains are not supported. **Multi-objective optimization** is not implemented. The NK landscape optimum computation uses exhaustive search, restricting its practical use to $n \leq 20$. Runtime profile analysis assumes integer-valued, unit-increment fitness functions for the exact tail-sum identity:

$$\mathbb{E}[\mathrm{CR}(T)] \;=\; \sum_{v=1}^{f^*} \sum_{t'=1}^{T} \Big[1 - P(\tau_v \leq t')\Big];$$

continuous fitness functions (e.g.`NK-Landscape`) yield approximate results and may exhibit minor discrepancies in the $\mathbb{E}[\mathrm{CR}(T)]$ verification plot.

3.1.5 Result Interpretations

All experiment output is written under a structured directory hierarchy. Raw JSON result files are stored at:

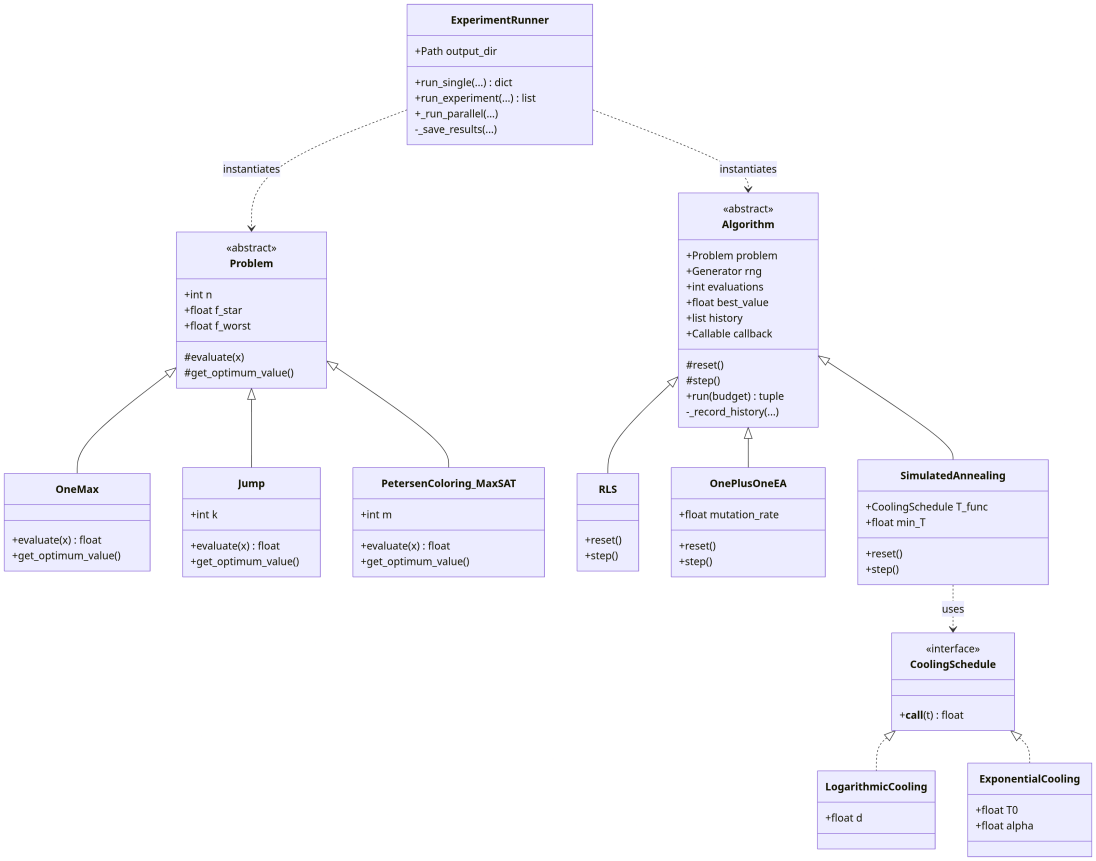
```
results/raw/<suite>/<problem>/<algorithm>/n<N>/b<B>.json
```

where `<suite>`, `<problem>`, and `<algorithm>` are normalized display names, `N` is the problem dimension, and `B` is the evaluation budget. Each file contains three top-level keys: `metadata` (algorithm, problem class, dimension, budget, run count, mode, and timestamp), `statistics` (mean, median, standard deviation, quartiles, and probability of optimum across all runs), and `results` (an ordered list of per-run dictionaries each containing regret, best value, optimality flag, evaluation count, seed, and the complete trajectory (in `full` mode)).

Generated figures are written under `results/figures/` following an analogous hierarchy:

```
results/figures/<suite>/<problem>/n<N>/<subdir>/<filename>.pdf
```

Figure 3.2: Simplified UML class diagram of the `regret` framework. Solid triangle arrows (▷) denote inheritance; dashed arrows denote dependency or instantiation. Hashes '#' indicate abstract declarations. Several concrete subclasses (e.g. `MuPlusLambdaEA`, `RLSExploration`, `NKLandscape`, `LinearCooling`) are omitted for clarity.



Plot subdirectories are configurable via `layout.structure` in the YAML configuration; the defaults are `aggregate/` for cross-budget plots, `history/` for trajectory and regret time-series plots, and `distributions/` for budget-specific box-plots and performance profiles. Runtime profile plots are written to a `profiles/` subdirectory within the raw results tree. All figures are saved as PDF at 300 DPI. Table 3.2 summarizes the full output organization.

Table 3.2: Output file organization for a representative experiment. "Normalization" replaces spaces, parentheses, and special characters with underscores or removes them.

Path pattern	**Contents**
Raw data (JSON)	
`results/raw/<suite>/<prob>/<algo>/n<N>/b<B>.json`	Per-budget result file: metadata, aggregate statistics, per-run regrets, and (in `full` mode) full evaluation trajectories.
`results/raw/<suite>/<prob>/profiles/n<N>/cr_profile_data_<algo>.csv`	Runtime profile CSV; columns: `evaluation`, `expected_cumulative_regret`, `mean_cumulative_regret`.
Figures (PDF, 300 DPI)	
`.../n<N>/aggregate/`	Cross-budget plots: regret curves, convergence probability, comparison heatmap.
`.../n<N>/history/`	Trajectory plots: best/current value, instantaneous and cumulative regret, Time to First Optimum (TTFO) distribution.
`.../n<N>/distributions/`	Budget-specific plots: regret box-plots and performance profiles.
`.../n<N>/profiles/`	Runtime profile plots: inverse profile surface/curves, $\mathbb{E}[\mathrm{CR}(T)]$ verification. Requires `suite.profile: true`.

Table 3.3: Plot types generated across experiments. "Profile required" denotes plots that require `suite.profile: true`

Plot	Output filename pattern - pdf format	Profile required
Regret curves	`regret_curves`	No
Convergence probability	`convergence_probability`	No
Comparison heatmap	`comparison_heatmap`	No
Regret box-plots	`regret_boxplot_b{budget}`	No
Performance profile	`performance_profile_b{budget}`	No
History (best-so-far)	`history_best`	No
Instantaneous regret	`history_regret_instantaneous`	No
Cumulative regret	`history_regret_cumulative`	No
TTFO distribution	`history_ttfo_markers`	No
Runtime profile surface	`inverse_runtime_profile_surface_{alg}`	Yes
Runtime profile curves	`inverse_runtime_profile_curves`	Yes
CR profile verification	`cr_profile_verification`	Yes

3.2 Testing strategy

The test suite follows a Test-Driven Development (TDD) discipline using a red–green–refactor workflow: tests were written prior to or alongside implementation and serve as executable specifications [1, 14]. Coverage is measured using the `coverage` [8] package with branch tracking enabled, and a minimum threshold of **50% branch coverage** is enforced in Continuous Integration (CI) via GitHub Actions. Coverage is also computed across Python 3.12, 3.13, and 3.14 in Continuous Integration (CI) for redundancy and support (the project must

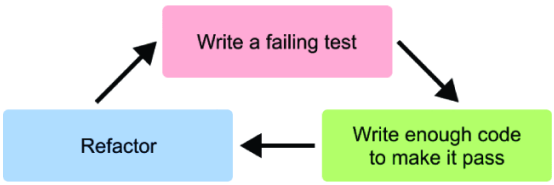


Figure 3.3: Red–Green–Refactor TDD technique. Start by writing a failing test (the red step). Next, write code that is just enough to make the written test pass (the green step). Finally, as the regularities and properties of the problem space are understood, refactors can be implemented that keeps the code clean (the blue step). The refactor step is safe since it will always be backed up by tests. [14]

be compatible for `python` $\geq$ 3.12).

Test organization follows a selective weighting strategy: core algorithmic components (`test_algorithms.py`, `test_metrics.py`, `test_problems.py`) and integration paths (`test_integration.py`) receive dense coverage, while infrastructure components such as CLI wiring and plot dispatch are covered at a coarser granularity. Shared fixtures in `conftest.py` provide canonical problem instances and bitstrings used across test suites.

3.3 Project Dependencies

The project is built with `hatchling` [31] and managed with `uv` [4], a Rust-based package and environment manager that replaces the Python standard `pip`/`venv`. Runtime installation: `uv sync`; development installation: `uv sync --group dev`.

Table 3.4: Package dependencies for the `regret` framework (Python $\geq$ 3.12).

Package	**Min. version**	**Purpose**
Scientific computing		
`numpy` [30]	2.4.3	Array operations, random number generation
`scipy` [58]	1.16.3	Statistics, drift computations, profile integrals
`pandas` [50]	2.3.3	Results aggregation, tabulation, CSV export
Visualisation		
`matplotlib` [35]	3.10.8	All static plot generation
`ipympl`	0.10.0	Interactive Matplotlib in notebooks
`networkx` [28]	3.6.1	Graph problem instances (e.g. Petersen colouring)
Configuration and CLI		
`pyyaml` [56]	6.0.3	Experiment configuration parsing
`jsonschema` [51]	4.26.0	YAML schema validation
`typer` [53]	0.24.1	`run_experiment` CLI
`tqdm` [17]	4.67.1	Progress bars for parallel runs
Notebook and interactive demonstration		
`jupyter` [27]	1.1.1	Notebook server
`ipykernel`	7.2.0	Jupyter kernel support
`ipywidgets`	8.1.8	Notebook UI widgets
Development — linting (optional)		
`ruff` [3]	—	Fast Rust-based linter and formatter (E, W, F, I, B, UP rules)
Development — testing (optional)		
`pytest` [42]	—	Unit and integration test runner
`coverage` [8]	—	Branch coverage reporting ($\geq$50% enforced)
`tabulate`	—	Tabular output in test fixtures
Development — type checking (optional)		
`ty` [5]	—	Rust-based static type checker (Astral)
Development — documentation (optional)		
`zensical`	—	Documentation site generator
`mkdocstrings-python`		Auto-generated API reference from docstrings

Chapter 4

Runtime Profiles and Anytime Performance

4.1 Motivation

Section 2.2.6 surveyed the three standard frameworks for measuring the performance of black-box optimization algorithms: the expected runtime $\mathbb{E}[\tau_{f^*}]$, fixed-budget analysis, and fixed-target analysis culminating in the runtime profile framework from [52]. Each improves on its predecessor in descriptive richness, yet none provides a single scalar that captures solution quality at every point in the run. Fixed-budget analysis, for instance, compresses the full trajectory to one value per budget [38]. What the practitioner actually observes is the evolution of solution quality over the *entire* course of the run.

Pinto and Doerr make the case for this richer mode of reporting directly:

> Our intention is rather to highlight to researchers working on theoretical aspects in evolutionary computation that, beyond being possibly more relevant for practitioners, explicitly reporting runtime profiles can give quite interesting insights into the performance of EAs. ([52], p. 19)

This chapter takes that framing as its starting point. Building on the notation established in Section 2.2.6.3, this chapter formalizes the distributional runtime profile as a mathematical object, and then proves that expected cumulative regret decomposes exactly as a double sum over this profile. The result unifies the runtime-analysis and regret-based perspectives into a single identity, providing

the *combined anytime measure* Pinto and Doerr [52] call for.

4.2 Variants

4.2.1 Expectation-Based Runtime Profiles

Building on the first hitting time τ_v defined in Section 2.2.6.3, Pinto and Doerr [52] propose reporting the full collection of expected first hitting times $\big(\mathbb{E}[\tau_v]\big)_v$ across all relevant target levels, the *expectation-based runtime profile*. When canonical fitness levels exist, such as for OneMax or LeadingOnes, these provide natural targets; for other functions, intermediate values identified during analysis or by linear interpolation of the fitness range may be used [52]. No new algorithms or proof techniques are required: the profile is typically already implicit in existing runtime proofs and calls only for a richer mode of presentation.

The practical value of this richer view is illustrated by the behavior of $(1+1)$ EA variants on LeadingOnes: reporting only the expected optimization time suggests RLS is superior, yet the runtime profile reveals that $(1+1)$ $\mathrm{EA}_{>0}$ (*App. para.* A.2) reaches all intermediate fitness levels $i \leq 7{,}980$ faster than RLS for problem size $n = 10{,}000$, with RLS only pulling ahead in the final portion of the run [52]. Such crossover behavior is directly relevant to the design of adaptive parameter and operator selection schemes.

4.2.2 Distributional Runtime Profiles

The expectation-based runtime profile of [52] summarizes the distribution of τ_v by its mean. A finer characterization, and the appropriate quantity for connecting runtime profiles to regret-based measures, is the *distributional runtime profile*

$$\mathcal{P}(v,t) \;:=\; \mathbb{P}(\tau_v > t), \tag{4.1}$$

which records, for each target level v and time step t, the probability that level v has not yet been reached by time t. The expectation-based profile is recovered by the standard tail-sum formula for non-negative integer-valued random variables [19, p. 16]:

$$\mathbb{E}[\tau_v] \;=\; \sum_{t=0}^{\infty} \mathbb{P}(\tau_v > t). \tag{4.2}$$

The distributional profile is the more primitive object from which expectation-based statements follow, and it constitutes the formal link between runtime profiles and the cumulative regret framework developed below.

Remark 2 (Naming convention)**.** *The term* runtime profile *appears in two distinct senses in the literature and in this work. [52] uses it to denote the* expectation-based *runtime profile* $\left(\mathbb{E}[\tau_v]\right)_v$*, which summarizes per-level performance by expected hitting times. The present work additionally employs the strictly finer* distributional *runtime profile* $\mathcal{P}(v,t) = \mathbb{P}(\tau_v > t)$*, from which the expectation-based profile is recovered via the tail-sum formula (Equation 4.2). To avoid ambiguity, both notions are referred to explicitly whenever both are in scope. Hereafter, and in all practical experimentation in this report,* runtime profile *refers to the distributional variant* $\mathcal{P}(v,t)$ *unless otherwise stated.*

4.3 Connection to Cumulative Regret

One of the directions suggested by Pinto and Doerr in their *Future Works* section is restated for emphasis:

> We feel that there is a need for combined measures that are capable of describing the anytime behavior of an EA. Regret-based measure as used in machine learning could be a key here, but we haven't been able so far to identify a fully satisfying measure. ([52], p. 22)

As far as we know, establishing such a relationship remains an open question. This section consolidates the following proposal: that the *Cumulative Regret (CR)* (Section 2.4.2.2) and the distributional runtime profile are not independent approaches to algorithm performance but are two representations of the same underlying quantity. The identity below attempts to establish this relation.

The relationship can be made explicit through a formal setup. A randomized black-box algorithm produces an adaptive sequence of query points $x_1, x_2, \ldots \in \{0,1\}^n$. Recall from Section 2.4.2.2 the *best-so-far process*

$$B(t) := \max_{s \le t} f(x_s),$$

which records the best objective value seen up to time t. Since $B(t)$ is non-decreasing in t, the event $\{B(t) < v\}$ is equivalent to $\{\tau_v > t\}$: the level v has not

yet been hit. The incumbent cumulative regret over a budget of T evaluations is therefore

$$\mathrm{CR}(T) := \sum_{t=1}^{T}\Big(f^* - B(t)\Big).$$

The proof of the main theorem relies on two elementary lemmas.

Lemma 1 (Tail-sum representation for the gap)**.** *For any integer b with $0 \le b \le f^*$,*

$$f^* - b = \sum_{v=1}^{f^*} \mathbf{1}[b < v].$$

Proof. The right-hand side counts the integers $v \in \{1, \dots, f^*\}$ that strictly exceed b, namely the set $\{b+1, \dots, f^*\}$, which has cardinality $f^* - b$. □

Lemma 2 (Indicator equivalence)**.** *For all $v \in \{1, \dots, f^*\}$ and $t \in \mathbb{N}$,*

$$\mathbf{1}[B(t) < v] = \mathbf{1}[\tau_v > t].$$

Proof. Since $B(t)$ is non-decreasing in t, the event $\{B(t) \ge v\}$ is equivalent to $\{\tau_v \le t\}$. Taking complements gives $\{B(t) < v\} = \{\tau_v > t\}$, so the two indicators agree on every sample path. □

The following identity connects *expected* cumulative regret to the distributional runtime profile.

Theorem 3 (Tail-Sum Regret Representation)**.** *For any randomized black-box algorithm operating on f with finite budget $T \in \mathbb{N}$,*

$$\mathbb{E}[\mathrm{CR}(T)] = \sum_{v=1}^{f^*}\sum_{t=1}^{T} \mathbb{P}(\tau_v > t).$$

Proof. For each t, Lemma 1 applied to $b = B(t)$ gives

$$f^* - B(t) = \sum_{v=1}^{f^*} \mathbf{1}[B(t) < v].$$

Summing over $t = 1, \dots, T$ yields

$$\mathrm{CR}(T) = \sum_{t=1}^{T}\Big(f^* - B(t)\Big) = \sum_{t=1}^{T}\sum_{v=1}^{f^*} \mathbf{1}[B(t) < v].$$

By Lemma 2, $\mathbf{1}[B(t) < v] = \mathbf{1}[\tau_v > t]$ on every sample path, so

$$\mathrm{CR}(T) \;=\; \sum_{t=1}^{T} \sum_{v=1}^{f^*} \mathbf{1}[\tau_v > t].$$

Taking expectations and applying linearity of expectation together with $\mathbb{E}[\mathbf{1}[\tau_v > t]] = \mathbb{P}(\tau_v > t)$ gives

$$\mathbb{E}[\mathrm{CR}(T)] \;=\; \sum_{t=1}^{T} \sum_{v=1}^{f^*} \mathbb{P}(\tau_v > t).$$

Since both sums are finite, their order may be exchanged, yielding the stated result. □

Theorem 3 expresses expected cumulative regret as a double sum of the distributional runtime profile: $\mathbb{E}[\mathrm{CR}(T)]$ is the total expected time the algorithm spends below each fitness level v, summed over all levels. An algorithm with a uniformly smaller runtime profile (one that reaches each fitness level earlier in expectation) therefore incurs lower cumulative regret. The regret measure thus simultaneously rewards early progress at every tier of the landscape, formalizing the intuition that steady improvement throughout a run is preferable to stagnation followed by a late breakthrough, even when both strategies share the same expected optimization time.

Taking the infinite-horizon limit, and applying the tail-sum identity for $\mathbb{E}[\tau_v]$ along with the monotone convergence theorem, yields the following asymptotic connection.

Corollary 4 (Asymptotic Regret and Expected Hitting Times)**.** *Suppose that τ_v is almost surely finite with finite expectation for each $v \in \{1, \ldots, f^*\}$. Then*

$$\lim_{T\to\infty} \mathbb{E}[\mathrm{CR}(T)] \;=\; \sum_{v=1}^{f^*} \mathbb{E}[\tau_v] \;-\; f^*.$$

Proof. **Step 1: Tail-sum identity for $\mathbb{E}[\tau_v]$.** Since $\tau_v \in \mathbb{N}_{\geq 1}$ by the convention that $t = 1$ is the first oracle call and initialization is excluded from the query budget, we have $\mathbb{P}(\tau_v > 0) = 1$. The standard tail-sum formula for a positive integer-valued random variable then gives

$$\mathbb{E}[\tau_v] \;=\; \sum_{t=0}^{\infty} \mathbb{P}(\tau_v > t) \;=\; 1 + \sum_{t=1}^{\infty} \mathbb{P}(\tau_v > t),$$

hence $\sum_{t=1}^{\infty} \mathbb{P}(\tau_v > t) = \mathbb{E}[\tau_v] - 1$.

Step 2: Passing to the limit in Theorem 3. For each fixed v, the partial sums $\sum_{t=1}^{T} \mathbb{P}(\tau_v > t)$ are non-decreasing in T and non-negative, so the monotone convergence theorem gives

$$\lim_{T\to\infty} \sum_{t=1}^{T} \mathbb{P}(\tau_v > t) \;=\; \sum_{t=1}^{\infty} \mathbb{P}(\tau_v > t) \;=\; \mathbb{E}[\tau_v] - 1.$$

Step 3: Summing over v. Applying Step 2 to each $v = 1, \ldots, f^*$ and invoking Theorem 3,

$$\lim_{T\to\infty} \mathbb{E}[\mathrm{CR}(T)] \;=\; \sum_{v=1}^{f^*} \Big(\mathbb{E}[\tau_v] - 1\Big) \;=\; \sum_{v=1}^{f^*} \mathbb{E}[\tau_v] \;-\; f^*. \qquad \square$$

Corollary 4 provides a unifying perspective: the long-run expected cumulative regret decomposes as the sum of the expected first hitting times over all fitness levels, minus the constant f^* arising from the $t = 0$ term in the tail-sum identity. Algorithms that reach all intermediate fitness levels quickly are therefore those that incur low long-run cumulative regret. This formalizes the intuition advanced in [52] that an algorithm making steady progress through the fitness landscape is preferable to one that stagnates at low fitness values even if both ultimately find the optimum in the same expected time.

The identity also reveals that the regret-based measures studied in the online learning literature [15] and the runtime profile framework of [52] are not independent approaches to measuring algorithm performance but are two views of the same underlying quantity, connected through the distributional runtime profile $\mathcal{P}(v, t)$.

4.4 Practical Relevance

As discussed in Section 2.2.6.4, the limitations of classical performance measures stem from their treatment of optimization as a sequence of threshold events rather than as a continuous quality trajectory.

Pinto and Doerr [52] note that runtime profiles are particularly valuable for the design of non-static parameter and operator selection strategies in EAs (such as adaptive mutation rates, crossover probabilities, and selection pressure) since such strategies must make decisions based on current solution quality rather than

on the expected optimization time alone. The identification of *crossover fitness levels* (where one algorithm overtakes another in the runtime profile) provides actionable guidance for when to switch between strategies.

Furthermore, as established in Theorem 3, any collection of upper bounds on $\mathbb{E}[\tau_v]$ for all levels v simultaneously yields an upper bound on expected cumulative regret, and conversely, lower bounds on regret propagate to lower bounds on a weighted aggregate of first hitting times. Cumulative regret thus provides a single scalar summary of the entire runtime profile, complementing the per-level view that the profile itself offers. The two representations (regret as a scalar, runtime profile as a function) are therefore best understood as complementary instruments for the same measurement task, connected by the identity of Theorem 3.

Chapter 5

Experiments

5.1 Experiment Structure

All experiments share a common infrastructure defined in the YAML configuration files under `configs/experiments/` !TODO: Cite appendices!. Each configuration specifies four top-level blocks: `suite` (global settings), `problems` (benchmark functions), `algorithms` (optimizers), and `plotting` (output).

5.1.1 Repetitions and budget

Each algorithm–problem pair is run for R independent repetitions (typically 25–100) at budgets drawn from a per-suite budget list. The display horizon `budget_for_plots` is chosen to be roughly twice the expected time-to-first-optimum of the hardest algorithm on each problem, so trajectory plots show both approach and post-convergence behavior.

5.1.2 Execution mode

All suites reported in this chapter were run in `full` mode with parallelism enabled. Full mode records complete per-evaluation trajectories, which are required for the instantaneous-regret, cumulative-regret, and TTFO analyses, and for all runtime-profile computations. Although the framework also supports lighter execution modes, those are outside the scope of the present experiments because they do not preserve the trajectory-level data needed for the chapter's main measurements.

5.1.3 Visualization and Table Strategy

Each suite produces plots and summary tables from the stored JSON results. Table 5.1 lists every plot type with its key, diagnostic purpose, and the suites or conditions that enable it.

Group	Output key	Diagnostic purpose	y-axis
Aggregate	`regret_curves`	Mean simple regret vs. budget; crossover budget between algorithms	log
	`convergence_probability`	$P(\text{opt})$ vs. budget; separates partial from full convergence	linear
	`comparison_heatmap`	$\log_{10}(\text{mean regret})$ colour-coded over all alg–budget cells	—
Distribution	`regret_boxplots`	Run-level spread of simple regret; skewness from non-converging runs	linear
	`performance_profile`	Empirical CDF of regret on log axis; steep rise = reliable convergence	log
Trajectory	`history_best`	Monotone incumbent fitness with f^* reference	linear
	`regret_instantaneous`	IR of evaluated solution; fluctuates when worsening moves accepted	log
	`regret_instantaneous_best`	IR of incumbent (monotone non-increasing); non-zero floor = saturation	log
	`regret_cumulative`	Integral of current-solution IR; penalises slow convergence	linear
	`regret_cumulative_best`	Integral of incumbent IR; key signal in barrier-crossing suites	linear

	`ttfo_distribution`	Jittered TTFO samples per algorithm; conditioned on convergence	linear
Runtime profile (suites 03, 08, 12)	`inverse_runtime_profile_surface`	$P(\tau_v \leq T)$ heat map over (T, v); saturation frontier visible	—
	`inverse_runtime_profile_curves`	$P(\tau_v \leq T)$ vs. T at $f^* \times \{0.25, 0.5, 0.75, 0.95\}$	linear
	`cr_profile_verification`	Overlay of direct and profile-derived $\mathbb{E}[\mathrm{CR}(T)]$	linear
Summary	per-problem table	Mean, median, SD, IQR, 95% CI, $P(\mathrm{opt})$, run count per algorithm	—
	aggregate table	Across-problem mean regret, mean $P(\mathrm{opt})$, best-count per algorithm	—

Table 5.1: Plot types produced by the experimental pipeline. The "Output key" column matches the YAML `plots` sub-block name. All trajectory and distribution plots are generated at `budget_for_plots`; aggregate plots pool across all budgets in the suite list. IR = instantaneous regret; CR = cumulative regret.

For hill-climbing algorithms (RLS, $(1+1)$-EA) the current-solution and incumbent IR variants coincide because no worsening moves are accepted. For SA and RLSExploration the two diverge during active exploration, making the current-solution variant a direct diagnostic of temperature-driven behaviour. Cumulative regret is always plotted on a linear y-axis: its slope during the trapped phase grows linearly with time, and a log axis would suppress that signal.

5.1.4 Suite Numbering and Roles

Suites are numbered 01–14. Table 5.2 gives a complete reference; the methodological context for runtime-profile suites (03, 08, 12) is developed in Chapter 4.

Suite	Role	Problems	Algorithms	R	Max budget	Profile
01	Baseline comparison	6 pseudo-boolean	RLS, RLSExpl, EA, SA-Log-d2.0	50	10 000	No
02	Extended benchmark	9 pseudo-boolean + combinatorial	same 4 families (with overrides)	50	20 000	No
08	Tail-sum verification	OneMax, LeadingOnes	RLS, $(1+1)$-EA	100	10 000	**Yes**
03	NK ruggedness (6×5 grid)	5 NK ($k \in \{1,2,4,6,8\}$, $n = 20$)	RLS, EA, 6×SA-Log	25	50 000	**Yes**
04	Trap: deception width × rate	4 Trap ($k \in \{5,8,10,20\}$)	4×EA + SA-Log-d4.0	25	10 000	No
05	Jump: mutation rate / plateau onset	5 Jump ($k \in \{2,3,5,7,10\}$)	3×EA (fixed + adaptive k/n)	25	20 000	No
06	Plateau: neutral walk × rate	4 Plateau ($k \in \{2,5,10,20\}$)	3×EA + SA-Log-d4.0	30	20 000	No
07	HIFF scaling	3 HIFF ($n \in \{32,64,128\}$)	RLS, 4×EA, SA-Log-d4.0	25	50 000	No
09	Jump: plateau duration (extended)	5 Jump (same as 05)	2×EA (fixed + adaptive)	25	50 000	No
10	Barrier archetype comparison	Jump/Trap/Plateau ($k = 5$, $n = 100$)	4×EA ($0.5/n$–$5/n$)	30	50 000	No
11	SA cooling diagnostics	OneMax, LeadingOnes	RLS, 6×SA-Log, SA-Lin, SA-Exp	50	10 000	No
12	NK saturation	5 NK ($k \in \{0,2,4,6,8\}$, $n = 20$)	RLS, $(1+1)$-EA, SA-Log-d4.0	25	50 000	**Yes**
13	Regret scaling with n	4 OneMax ($n \in \{25,50,100,200\}$)	$(1+1)$-EA at rate $1/n$	60	10 000	No
14	Population dynamics	OneMax, LeadingOnes, TwoMax, Jump-3	$(1+1)$-EA + 4×$(\mu+\lambda)$-EA	30	10 000	No

Table 5.2: Overview of all 14 experimental suites. R = runs per algorithm–problem–budget combination. "Profile" indicates whether `suite.profile: true` is set. Budget values are in evaluations; "max" is the largest value in the suite budget list.

5.2 Baseline Suites

5.2.1 Suite 01: Algorithm-Family Comparison

Suite 01 establishes a reference comparison of four algorithm families across six pseudo-boolean problems of increasing structural difficulty. The four families are: Randomized Local Search (RLS), RLSExploration, the $(1+1)$-EA, and SA with logarithmic cooling at $d = 2.0$ (SA-Log-d_2.0). SA is reduced to a single representative schedule to preserve a clean four-family distinction; the effect of varying the schedule parameter is deferred to suites 03 and 11. All algorithm entries carry empty `by_problem` blocks, so the same configuration applies across all six problems.

The six problems are: OneMax ($n = 100$), LeadingOnes ($n = 100$), TwoMax ($n = 100$), Jump-3 ($n = 100$, $k = 3$), Trap-10 ($n = 100$, $k = 10$), and BinVal ($n = 50$). BinVal uses $n = 50$ rather than $n = 100$: its exponential fitness structure ($f^* = 2^{50} - 1 \approx 10^{15}$) already produces large fitness gaps and rapid convergence at this size, and extending to $n = 100$ would make the optimum value $2^{100} - 1$ while adding no structural insight. The much smaller `budget_for_plots` value of 500 for BinVal, compared with 2000–10 000 for the other problems, reflects the expectation that convergence occurs early.

The 50 runs per combination are evaluated at six budgets: $\{200, 500, 1000, 2000, 5000, 10000\}$. The upper budget of 10 000 is chosen to include the regime where LeadingOnes and Jump-3 actually converge for some algorithms; the lower bound of 200 captures the very-early transient. Pairwise statistical annotations are enabled on regret box-plots, performance profiles, and TTFO distributions, using RLS as the reference algorithm.

5.2.2 Suite 02: Extended Pseudo-Boolean Benchmark

Suite 02 extends the baseline to nine problems, adding Jump-5, Trap-5, Plateau-10, HIFF-64, and a combinatorial instance (Petersen-3-Color). The algorithm set is identical to suite 01 (the same four families). Budgets range from 500 to 20 000 over 50 runs. Profile analysis is disabled.

Table 5.3 details the problem-specific hyperparameter overrides applied in this suite. The override problems — Jump-3, Jump-5, and Trap-5 — share the structural property of having barriers that require sustained exploration; the combinatorial instance and the unimodal and bimodal problems receive no overrides.

For RLSExploration, fixing $\varepsilon = 0.05$ and disabling decay holds the exploration rate constant at five times the default, maintaining persistent non-improving moves on problems where barrier crossing requires sustained exploration. For SA, tripling d under

Table 5.3: Problem-specific algorithm overrides in suite 02. For RLSExploration the default is $\varepsilon = 1/n = 0.01$ with exponential decay ($\varepsilon_t = \varepsilon/t$); for SA the default cooling parameter is $d = 2.0$.

Algorithm	**Problem**	**Parameter**	**Default**	**Override**
RLSExploration	Jump-3	ε, decay	0.01, on	0.05, off
	Jump-5	ε, decay	0.01, on	0.05, off
	Trap-5	ε, decay	0.01, on	0.05, off
SA-Log-d_2.0	Jump-5	d	2.0	6.0
	Trap-5	d	2.0	6.0

logarithmic cooling $T(t) = d/\log(t+1)$ triples the temperature at every evaluation index, raising acceptance probability for worsening moves throughout the run. Both modifications target the same function: improving escape from local or deceptive optima at moderate budget.

5.3 Verification Suite

5.3.1 Suite 08: Runtime Profile Verification

Suite 08 validates the tail-sum identity

$$\mathbb{E}[\mathrm{CR}(T)] = \sum_{v=1}^{f^*} \sum_{t'=1}^{T} [1 - P(\tau_v \leq t')]$$

on two problems with well-understood convergence behavior: OneMax and LeadingOnes ($n = 100$), using only RLS and the $(1+1)$-EA.

The suite is configured with 100 independent runs, more than any other suite. This is not for statistical power in the usual sense (the goal is not to detect a treatment effect) but to suppress Monte-Carlo variance in the empirical mean $\widehat{\mathbb{E}}[\mathrm{CR}(T)]$ to the point where any visible gap between the trajectory-derived and profile-derived curves in the verification plot can be attributed to numerical discretization rather than sampling noise. With $R = 100$ runs and the integer-grid profile implementation (exact for unit-increment fitness), any remaining discrepancy indicates a bug in trajectory recording, cumulative-regret computation, or profile construction.

Profile analysis is enabled (`profile: true`). The `cr_profile_verification` plot is the primary output: it overlays the two derivations of $\mathbb{E}[\mathrm{CR}(T)]$ for each algorithm on a shared time axis. Inverse runtime profile surface and curve plots are also produced. The `budget_for_plots` for LeadingOnes is set to 10 000, which is the largest available

budget; this is set deliberately to capture the full cumulative regret ramp, which extends further than on OneMax due to the $\Theta(n^2)$ expected convergence time.

Pairwise annotations are enabled on box-plots and TTFO distributions with RLS as the reference, though these are secondary outputs given the verification focus.

5.4 Exploratory Analysis of Regret and Runtime

The exploratory suites address four themes: barrier structure and mutation rate (suites 04–06, 09–10), NK ruggedness and SA cooling (suites 03, 11–12), hierarchical structure and scaling (suites 07, 13), and population dynamics (suite 14). Their purpose is to examine how regret and convergence behaviour change with landscape structure, cooling schedule, problem size, and population configuration.

5.4.1 Suite 03: NK Landscape Ruggedness

Suite 03 crosses six logarithmic SA schedules ($d \in \{0.5, 1.0, 2.0, 4.0, 8.0, 16.0\}$) with five NK instances ($k \in \{1, 2, 4, 6, 8\}$, $n = 20$, seed 42). RLS and the $(1+1)$-EA are included as zero-temperature baselines so that the effect of temperature-driven acceptance can be separated from plain hill-climbing.

All SA entries use empty `by_problem` blocks, so the suite forms a clean 6×5 grid over cooling parameter and epistasis level without per-problem tuning. The restriction to $n = 20$ is imposed by the exact-optimum computation used in the NK implementation. Profile analysis is enabled, producing inverse runtime profile surfaces, selected profile curves, and the profile verification plot. The extended budget to 50 000 is needed because high-epistasis instances may approach saturation only late in the run.

5.4.2 Suite 04: Trap Function Mutation Rate

Suite 04 evaluates Trap with four $(1+1)$-EA mutation rates ($0.5/n$, $1/n$, $2/n$, $5/n$) and one SA variant across four deception widths ($k \in \{5, 8, 10, 20\}$, $n = 100$). The EA rates are fixed across all problems. SA-Log-d_4.0 uses $d = 4.0$ by default, with a single override to $d = 8.0$ on Trap-k20.

Trap-k1 is excluded because it yields uniformly uninformative behaviour: all algorithms are driven to the deceptive attractor, so the resulting plots are essentially flat. The chosen EA rates are shared with suites 05 and 06 to support cross-suite comparison of regret shape across barrier types. Profile analysis is disabled.

5.4.3 Suite 05: Jump Function Mutation Rate

Suite 05 studies five Jump instances ($k \in \{2,3,5,7,10\}$, $n = 100$) under three $(1+1)$-EA variants: fixed rates $1/n$ and $2/n$, and a problem-adaptive rate k/n implemented through per-problem overrides.

The lower rate $0.5/n$ is omitted because it offers no advantage for crossing a k-bit gap relative to $1/n$. Budgets range from 500 to 20 000. Instantaneous regret is plotted on a log scale, while cumulative regret is kept on a linear scale so that the trapped-phase slope remains visible. Profile analysis is disabled; the longer-horizon Jump study is deferred to suite 09.

5.4.4 Suite 06: Plateau Function Mutation Rate

Suite 06 mirrors suite 05 on Plateau instances ($k \in \{2,5,10,20\}$, $n = 100$) to compare cumulative regret trajectory shape across barrier types at structurally equivalent widths. The algorithm set is three $(1+1)$-EA variants at rates $1/n$, $2/n$, and $5/n$, plus SA-Log-d_4.0 with no per-problem overrides.

The adaptive k/n rate from suite 05 is omitted: it was motivated by the need to maximize the probability of exactly k simultaneous flips for the Jump gap, a rationale that has no equivalent for the neutral walk on Plateau, which does not require any specific number of simultaneous flips. SA-Log-d_4.0 is added instead as a third algorithm class to assess whether temperature-driven acceptance confers any advantage on a landscape where the neutral zone carries no fitness penalty. The shared values $k = 2$ and $k = 5$ support direct comparison with suite 05; $k = 10$ and $k = 20$ extend the plateau duration beyond what can converge within the 20 000-evaluation budget. Profile analysis is disabled.

5.4.5 Suite 07: Hierarchical IFF Scaling

Suite 07 evaluates HIFF at $n \in \{32,64,128\}$ under RLS, four $(1+1)$-EA variants, and SA-Log-d_4.0 at a fixed $d = 4.0$. RLS is included as a zero-temperature reference whose rapid saturation on large instances anchors the comparison: a single bit-flip disrupts any uniform sub-block, so RLS is expected to stagnate at the first hierarchical level for $n \geq 64$. Table 5.4 shows the exact mutation rate values used for each EA variant at each problem size.

For $n = 128$, `budget_for_plots` is set to 50 000, the largest budget in the suite. Convergence is not expected within this budget; the setting is deliberate, making the unsolved $n = 128$ trajectory an informative contrast to the partially or fully solved $n = 32$ and $n = 64$ cases on the same plot axes. Profile analysis is disabled.

Table 5.4: Per-problem mutation rates in suite 07. Values are set via `by_problem` overrides to exact decimal representations of $\{0.5/n, 1/n, 2/n, 3/n\}$ at each problem size.

Algorithm	$n = 32$	$n = 64$	$n = 128$
EA-mu_0.5-by-n	$1/64 \approx 0.01563$	$1/128 \approx 0.00781$	$1/256 \approx 0.00391$
EA-mu_1-by-n	$1/32 \approx 0.03125$	$1/64 \approx 0.01563$	$1/128 \approx 0.00781$
EA-mu_2-by-n	$2/32 \approx 0.06250$	$2/64 \approx 0.03125$	$2/128 \approx 0.01563$
EA-mu_3-by-n	$3/32 \approx 0.09375$	$3/64 \approx 0.04688$	$3/128 \approx 0.02344$

5.4.6 Suite 09: Jump Plateau Duration

Suite 09 uses the same five Jump instances and the same two EA variants as suite 05 (rates $1/n$ and k/n with identical per-problem overrides), but differs in its budget configuration. The budget list is $\{1000, 5000, 10000, 25000, 50000\}$, dropping the 500-evaluation point from suite 05 and extending the maximum to 50 000. The `budget_for_plots` values are also extended: Jump-k2 uses 10 000, Jump-k3 uses 25 000, and Jump-k5, Jump-k7, Jump-k10 each use 50 000. In suite 05, Jump-k5 through Jump-k10 used 20 000.

The difference between the two suites is therefore one of time horizon rather than algorithm or problem design. Suite 05 captures plateau onset (the transition from hill-climbing to the trapped phase) for the smaller k values; suite 09 extends the horizon to capture the plateau duration and, for small k at the adaptive rate, potentially the plateau exit as well.

Profile analysis is disabled.

5.4.7 Suite 10: Barrier Structure Comparison

Suite 10 compares Jump-k5, Trap-k5, and Plateau-k5 at $n = 100$ under four $(1+1)$-EA mutation rates: $0.5/n$, $1/n$, $2/n$, and $5/n$. The same rates are used on all three barrier types, and the budget extends to 50 000.

The suite is designed to compare how regret accumulates under barriers of equal nominal width but different structure. TTFO alone is often too coarse in this regime, whereas cumulative regret can distinguish early deception, delayed barrier crossing, and neutral trapping. Pairwise annotations use `(1+1)-EA-mu_1-by-n` as the reference algorithm. Profile analysis is disabled.

5.4.8 Suite 11: SA Cooling Schedule Diagnostics

Suite 11 evaluates eight SA variants and one RLS baseline on OneMax and LeadingOnes ($n = 100$). The SA set consists of six logarithmic schedules ($d \in \{0.5, 1.0, 2.0, 4.0, 8.0, 16.0\}$), one linear schedule (SA-Lin), and one exponential schedule (SA-Exp), all applied uniformly across both problems with no per-problem overrides.

The linear and exponential variants use implementation defaults: linear cooling with $T_0 = 1.0$, $T_f = 0.01$, max_iter $= 10\,000$; exponential cooling with $T_0 = 1.0$, $\alpha = 0.95$. Both schedules reach their minimum temperature within the available budget (linear exactly at $t = 10\,000$, exponential effectively by $t \approx 90$ since $0.95^{90} \approx 0.01$) after which SA reduces to a near-deterministic hill-climber. Logarithmic schedules, by contrast, remain strictly positive for all finite t. This structural distinction is the primary hypothesis the suite is designed to expose as a signature in the instantaneous regret trajectory. Pairwise annotations use RLS as the reference so that any SA configuration underperforming plain hill-climbing is directly identifiable in the annotated plots. Profile analysis is disabled.

5.4.9 Suite 12: NK Saturation

Suite 12 examines five NK landscape instances ($k \in \{0, 2, 4, 6, 8\}$, $n = 20$, seed 42) under three algorithms: RLS, the $(1+1)$-EA, and SA-Log-d_4.0. The NK instances use the same fixed seed as suite 03, making the landscape realisations identical across both suites for the shared k values.

Suite 12 differs from suite 03 in two structural ways. First, it includes NK-k0 ($k = 0$, fully separable), absent from suite 03. At $k = 0$ each locus contributes independently to fitness, providing a baseline against which saturation at higher k can be measured. Second, the SA cooling parameter escalates with k via `by_problem` overrides (Table 5.5), reflecting that deeper local optima at higher epistasis require higher temperature to maintain non-trivial acceptance probability. This tuning means suite 12 does not form a clean grid design for studying the SA cooling–epistasis interaction (that role belongs to suite 03), but gives the SA variant a reasonable operating point at each k to assess whether adequate temperature maintenance changes the saturation outcome.

Table 5.5: SA-Log cooling parameter d per NK instance in suite 12. Values increase monotonically with epistasis k to compensate for deeper local optima.

NK instance	NK-k0	NK-k2	NK-k4	NK-k6	NK-k8
d	4.0	4.0	8.0	8.0	12.0

Profile analysis is enabled, producing inverse runtime profile surfaces and curves.

The `cr_profile_verification` plot is not enabled in this suite: the NK landscape uses a continuous fitness function normalized to $[0,1]$, so the integer-grid profile implementation does not apply exactly.

5.4.10 Suite 13: Regret Scaling with Problem Size

Suite 13 runs a single algorithm, $(1+1)$-EA at rate $1/n$, on four OneMax instances with $n \in \{25, 50, 100, 200\}$. The mutation rates are set via per-problem overrides to exact values $\{0.04, 0.02, 0.01, 0.005\}$ corresponding to $1/n$ at each size; the `defaults` block carries 0.01 as a fallback. Using a single algorithm removes algorithm-to-algorithm comparisons from the design and focuses the suite entirely on the scaling behavior of the regret metric across problem sizes. With 60 runs (the highest run count after suite 08) the empirical $\mathbb{E}[\mathrm{CR}(T)]$ estimates are suitable for fitting a scaling exponent.

The `budget_for_plots` values are $\{1000, 2500, 5000, 10000\}$ for $n \in \{25, 50, 100, 200\}$ respectively. These are approximately consistent with $5n \log_2 n$ (giving roughly 580, 1411, 3322, and 7640), but set somewhat higher to ensure that the full post-convergence plateau is also visible in the trajectory plots. The budget list $\{100, 500, 1000, 2500, 5000, 10000\}$ includes a very short 100-evaluation point, reflecting the interest in early transient behavior at small n.

Profile analysis is disabled.

5.4.11 Suite 14: Population Dynamics

Suite 14 compares the standard $(1+1)$-EA against four $(\mu+\lambda)$-EA variants on four problems: OneMax, LeadingOnes, TwoMax, and Jump-3 ($n = 100$). All five algorithm entries use mutation rate $1/n = 0.01$ with no per-problem overrides, isolating the effect of population structure from mutation rate.

The five configurations are: $(1+1)$-EA (the standard single-individual baseline via its own class); $(1+1)$-EA-framework ($\mu = 1,\ \lambda = 1$ via the MuPlusLambdaEA class); $(5+5)$-EA ($\mu = 5,\ \lambda = 5$); $(10+20)$-EA ($\mu = 10,\ \lambda = 20$); and $(20+10)$-EA ($\mu = 20,\ \lambda = 10$). Including both $(1+1)$-EA and $(1+1)$-EA-framework is deliberate: the two entries use different implementation classes but the same logical algorithm, so any difference in their results would indicate a framework overhead or implementation discrepancy rather than a population effect.

The $(\mu+\lambda)$-EA implementation records one history entry per offspring evaluation rather than per generation, which was the reason for its exclusion from suite 02. In suite 14 all algorithms are either the $(1+1)$-EA or $(\mu+\lambda)$-EA variants, so the recording granularity is consistent within the suite. The four problems span unimodal (OneMax),

sequentially structured (LeadingOnes), bimodal (TwoMax), and gap-structured (Jump-3) fitness landscapes, providing a range of settings in which the cost and benefit of maintaining a population may differ.

Profile analysis is disabled.

5.5 Cross-Suite Design Relationships

Several structural relationships between suites affect which results can be meaningfully compared. Table 5.6 summarizes the three principal links; the paragraphs below elaborate the design rationale.

Table 5.6: Principal cross-suite design relationships.

Relationship	Suites	Shared	Differs
Barrier types at equal rates	04, 05, 06, 10	EA rates $1/n$ and $2/n$; problems at same nominal k	Barrier type (Trap/Jump/Plateau); additional rates per suite
Jump at two time horizons	05, 09	All 5 Jump instances; EA variants at $1/n$ and k/n	Budget list and `budget_for_plots` (onset vs. duration)
NK at two SA designs	03, 12	$n = 20$, seed 42; shared $k \in \{2,4,6,8\}$; identical landscapes	Suite 03: 6×5 d-grid, no tuning; Suite 12: per-k d escalation, adds $k = 0$

Barrier types across suites 04–06 and 10. Suites 04, 05, and 06 share the fixed rates $1/n$ and $2/n$. The full rate sets differ: suite 04 uses $\{0.5/n, 1/n, 2/n, 5/n\}$; suite 05 uses $\{1/n, 2/n\}$ plus the adaptive k/n (with $0.5/n$ omitted as dominated and $5/n$ absent); suite 06 uses $\{1/n, 2/n, 5/n\}$ (with $0.5/n$ absent for the same reason). At the shared rates, results on Trap-k, Jump-k, and Plateau-k at the same k are directly comparable since the algorithm is otherwise identical. Suite 10 consolidates this comparison by running all four EA rates on all three barrier types simultaneously at $k = 5$.

Jump across suites 05 and 09. Suites 05 and 09 are identical in problem and algorithm configuration; they differ only in budget list and `budget_for_plots`. Suite 05 is oriented toward observing plateau onset at small k; suite 09 extends the horizon

to study plateau duration at all k values, including those where convergence within budget is unlikely.

NK across suites 03 and 12. Both suites use $n = 20$, seed 42. Suite 03 covers $k \in \{1, 2, 4, 6, 8\}$ and suite 12 covers $k \in \{0, 2, 4, 6, 8\}$; the shared values are $k \in \{2, 4, 6, 8\}$, and the landscape realisations at those values are identical. Suite 03 forms a clean grid over d values with no per-problem tuning, making any cooling–epistasis interaction attributable to problem structure rather than instance-specific calibration. Suite 12 uses per-k d escalation and adds the $k = 0$ separable baseline, providing a different perspective on the saturation phenomenon.

Chapter 6

Evaluation & Discussion of Results

6.1 Overview

6.2 Baseline Suites

6.2.1 Suite 01: Algorithm-Family Comparison

6.2.1.1 Aggregate Regret and Convergence

6.2.1.2 Regret Distributions and TTFO

6.2.1.3 Trajectory Analysis

6.2.2 Suite 02: Extended Pseudo-Boolean Benchmark

6.2.2.1 Aggregate Regret and Convergence

6.2.2.2 Effect of Algorithm Overrides

6.2.2.3 Trajectory Analysis

6.3 Verification Suite

6.3.1 Suite 08: Runtime Profile Verification

6.3.1.1 Tail-Sum Identity: Direct vs. Profile-Derived $\mathbb{E}[\mathrm{CR}(T)]$

6.3.1.2 Inverse Runtime Profile Surfaces and Curves

6.4 Exploratory Analysis

6.4.1 Barrier Structure and Mutation Rate

6.4.1.1 Suite 04: Trap – Deception Width and Mutation Rate

Chapter 7

Conclusion & Future Work

7.1 Conclusion

7.2 Future Work

Bibliography

[1] Dua Agha, Rashida Sohail, Areej Fatemah Meghji, Ramsha Qaboolio, and Sania Bhatti. Test driven development and its impact on program design and software quality: A systematic literature review. *VAWKUM Transactions on Computer Sciences*, 11(1):268–280, 2023.

[2] Denis Antipov and Benjamin Doerr. Precise runtime analysis for plateau functions. *ACM Trans. Evol. Learn. Optim.*, 1(4), October 2021. ISSN 2688-299X. doi: 10.1145/3469800. URL https://doi.org/10.1145/3469800.

[3] Astral. Ruff: An extremely fast Python linter and code formatter, 2022. URL https://github.com/astral-sh/ruff. Accessed: 2026.

[4] Astral. uv: An extremely fast Python package and project manager, 2024. URL https://github.com/astral-sh/uv. Accessed: 2026.

[5] Astral. ty: A fast Python type checker and language server, 2025. URL https://github.com/astral-sh/ty. Accessed: 2026.

[6] Charles Audet and Warren Hare. *Derivative-free and blackbox optimization*, volume 1. Springer, 2017.

[7] Peter Auer, Nicolo Cesa-Bianchi, and Paul Fischer. Finite-time analysis of the multiarmed bandit problem. *Machine learning*, 47(2):235–256, 2002.

[8] Ned Batchelder and contributors. coverage.py: Code coverage measurement for Python, 2009. URL https://github.com/nedbat/coveragepy. Accessed: 2026.

[9] Dimitri Bertsekas. *Convex optimization theory*, volume 1. Athena Scientific, 2009.

[10] Dimitris Bertsimas and John Tsitsiklis. Simulated annealing. *Statistical science*, 8(1):10–15, 1993.

[11] Süntje Böttcher, Benjamin Doerr, and Frank Neumann. Optimal fixed and adaptive mutation rates for the leadingones problem. In *International Conference on Parallel Problem Solving from Nature*, pages 1–10. Springer, 2010.

[12] Sébastien Bubeck and Nicolo Cesa-Bianchi. Regret analysis of stochastic and nonstochastic multi-armed bandit problems. *arXiv preprint arXiv:1204.5721*, 2012.

[13] Sébastien Bubeck, Rémi Munos, and Gilles Stoltz. Pure exploration in finitely-armed and continuous-armed bandits. *Theoretical Computer Science*, 412(19): 1832–1852, 2011.

[14] Pedro Calais and Lissa Franzini. Test-driven development benefits beyond design quality: Flow state and developer experience. In *2023 IEEE/ACM 45th International Conference on Software Engineering: New Ideas and Emerging Results (ICSE-NIER)*, pages 106–111, 2023. doi: 10.1109/ICSE-NIER58687.2023.00025.

[15] Nicolo Cesa-Bianchi and Gábor Lugosi. *Prediction, learning, and games*. Cambridge university press, 2006.

[16] Sayak Ray Chowdhury and Aditya Gopalan. On kernelized multi-armed bandits. In *International Conference on Machine Learning*, pages 844–853. PMLR, 2017.

[17] Casper da Costa-Luis, Stephen Karl Larroque, Kyle Altendorf, Hadrien Mary, richardsheridan, Mikhail Korobov, Noam Raphael, Ivan Ivanov, Marcel Bargull, Nishant Rodrigues, Shawn, Mikhail Dektiarev, Michał Górny, mjstevens777, Matthew D. Pagel, Martin Zugnoni, CrazyPython, Charles Newey, Antony Lee, pgajdos, Todd, Staffan Malmgren, redbug312, Orivej Desh, Nikolai Nechaev, Mike Boyle, Max Nordlund, MapleCCC, Jack McCracken, and Hugo van Kemenade. tqdm: A fast, extensible progress bar for python and cli, February 2026. URL https://doi.org/10.5281/zenodo.18473238.

[18] Kalyanmoy Deb and David E Goldberg. Analyzing deception in trap functions. In *Foundations of genetic algorithms*, volume 2, pages 93–108. Elsevier, 1993.

[19] Benjamin Doerr and Frank Neumann. Theory of evolutionary computation. *Recent Developments in Discrete Optimization. Springer, Cham*, 2020.

[20] Carola Doerr and Johannes Lengler. Onemax in black-box models with several restrictions. In *Proceedings of the 2015 Annual Conference on Genetic and Evolutionary Computation*, pages 1431–1438, 2015.

[21] Stefan Droste, Thomas Jansen, and Ingo Wegener. On the analysis of the (1+ 1) evolutionary algorithm. *Theoretical Computer Science*, 276(1-2):51–81, 2002.

[22] Stefan Droste, Thomas Jansen, and Ingo Wegener. Upper and lower bounds for randomized search heuristics in black-box optimization. *Theory of computing systems*, 39(4):525–544, 2006.

[23] Elias Freider Erik Bernhardsson. Luigi, 2012. URL https://github.com/python-jsonschema/jsonschema. Accessed: 2026.

[24] Peter I Frazier. A tutorial on bayesian optimization. *arXiv preprint arXiv:1807.02811*, 2018.

[25] Benyamin Ghojogh, Ali Ghodsi, Fakhri Karray, and Mark Crowley. Reproducing kernel hilbert space, mercer's theorem, eigenfunctions, nystr\" om method, and use of kernels in machine learning: Tutorial and survey. *arXiv preprint arXiv:2106.08443*, 2021.

[26] Daniel Golovin, Benjamin Solnik, Subhodeep Moitra, Greg Kochanski, John Karro, and D. Sculley. Google vizier: A service for black-box optimization. In *Proceedings of the 23rd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, Halifax, NS, Canada, August 13 - 17, 2017*, pages 1487–1495. ACM, 2017. doi: 10.1145/3097983.3098043. URL https://doi.org/10.1145/3097983.3098043.

[27] Brian E. Granger and Fernando Pérez. Jupyter: Thinking and storytelling with code and data. *Computing in Science & Engineering*, 23(2):7–14, 2021. doi: 10.1109/MCSE.2021.3059263.

[28] Aric A. Hagberg, Daniel A. Schult, and Pieter J. Swart. NetworkX, 2008. URL https://networkx.org. Accessed: 2026.

[29] Bruce Hajek. Cooling schedules for optimal annealing. *Mathematics of operations research*, 13(2):311–329, 1988.

[30] Charles R. Harris, K. Jarrod Millman, Stéfan J. van der Walt, Ralf Gommers, Pauli Virtanen, David Cournapeau, Eric Wieser, Julian Taylor, Sebastian Berg, Nathaniel J. Smith, Robert Kern, Matti Picus, Stephan Hoyer, Marten H. van Kerkwijk, Matthew Brett, Allan Haldane, Jaime Fernández del Río, Mark Wiebe, Pearu Peterson, Pierre Gérard-Marchant, Kevin Sheppard, Tyler Reddy, Warren Weckesser, Hameer Abbasi, Christoph Gohlke, and Travis E. Oliphant. Array

programming with NumPy. *Nature*, 585(7825):357–362, September 2020. doi: 10.1038/s41586-020-2649-2. URL https://doi.org/10.1038/s41586-020-2649-2.

[31] Hatch Developers. Hatchling: The build backend for the Hatch project manager, 2022. URL https://hatch.pypa.io/latest/. Accessed: 2026.

[32] Holger H. Hoos and Thomas Stützle. 1 - introduction. In Holger H. Hoos and Thomas Stützle, editors, *Stochastic Local Search*, The Morgan Kaufmann Series in Artificial Intelligence, pages 13–59. Morgan Kaufmann, San Francisco, 2005. ISBN 978-1-55860-872-6. doi: https://doi.org/10.1016/B978-155860872-6/50018-4. URL https://www.sciencedirect.com/science/article/pii/B9781558608726500184.

[33] Holger H. Hoos, Kevin Smyth, and Thomas Stützle. Search space features underlying the performance of stochastic local search algorithms for MAX-SAT. In Xin Yao, Edmund K. Burke, José A. Lozano, Jim Smith, Juan Julián Merelo-Guervós, John A. Bullinaria, Jonathan E. Rowe, Peter Tiňo, Ata Kabán, and Hans-Paul Schwefel, editors, *Parallel Problem Solving from Nature - PPSN VIII*, volume 3242, pages 51–60. Springer Berlin Heidelberg, 2004. ISBN 978-3-540-23092-2 978-3-540-30217-9. doi: 10.1007/978-3-540-30217-9_6. URL http://link.springer.com/10.1007/978-3-540-30217-9_6. Series Title: Lecture Notes in Computer Science.

[34] Beichen Huang, Ran Cheng, Zhuozhao Li, Yaochu Jin, and Kay Chen Tan. EvoX: A Distributed GPU-accelerated Framework for Scalable Evolutionary Computation. *IEEE Transactions on Evolutionary Computation*, 2024. doi: 10.1109/TEVC.2024.3388550.

[35] John D Hunter. Matplotlib: A 2d graphics environment. *Computing in science & engineering*, 9(3):90–95, 2007.

[36] Hsien-Kuei Hwang, Alois Panholzer, Nicolas Rolin, Tsung-Hsi Tsai, and Wei-Mei Chen. Probabilistic analysis of the (1+ 1)-evolutionary algorithm. *Evolutionary computation*, 26(2):299–345, 2018.

[37] Thomas Jansen and Christine Zarges. Analysis of evolutionary algorithms: from computational complexity analysis to algorithm engineering. In *Proceedings of the 11th workshop proceedings on Foundations of genetic algorithms*, pages 1–14, 2011.

[38] Thomas Jansen and Christine Zarges. Performance analysis of randomised search heuristics operating with a fixed budget. *Theoretical Computer Science*, 545:39–58, 2014.

[39] Donald R Jones, Matthias Schonlau, and William J Welch. Efficient global optimization of expensive black-box functions. *Journal of Global optimization*, 13(4): 455–492, 1998.

[40] Stuart Kauffman and Simon Levin. Towards a general theory of adaptive walks on rugged landscapes. *Journal of theoretical Biology*, 128(1):11–45, 1987.

[41] Scott Kirkpatrick, C Daniel Gelatt Jr, and Mario P Vecchi. Optimization by simulated annealing. *science*, 220(4598):671–680, 1983.

[42] Holger Krekel, Bruno Oliveira, Ronny Pfannschmidt, Floris Bruynooghe, Brianna Laugher, and Freya Bruhin. pytest x.y, 2004. URL https://github.com/pytest-dev/pytest. Contributors: Holger Krekel and Bruno Oliveira and Ronny Pfannschmidt and Floris Bruynooghe and Brianna Laugher and Freya Bruhin and others.

[43] Tze Leung Lai and Herbert Robbins. Asymptotically efficient adaptive allocation rules. *Advances in applied mathematics*, 6(1):4–22, 1985.

[44] Tor Lattimore and Csaba Szepesvári. *Bandit algorithms*. Cambridge University Press, 2020.

[45] Johannes Lengler. Drift analysis. In *Theory of evolutionary computation: Recent developments in discrete optimization*, pages 89–131. Springer, 2019.

[46] Sijia Liu, Pin-Yu Chen, Bhavya Kailkhura, Gaoyuan Zhang, Alfred O. Hero, and Pramod K. Varshney. A Primer on Zeroth-Order Optimization in Signal Processing and Machine Learning: Principals, Recent Advances, and Applications. *IEEE Signal Processing Magazine*, 37(5):43–54, September 2020. ISSN 1053-5888, 1558-0792. doi: 10.1109/MSP.2020.3003837. URL https://ieeexplore.ieee.org/document/9186148/.

[47] F Mölder, KP Jablonski, B Letcher, MB Hall, PC van Dyken, CH Tomkins-Tinch, V Sochat, J Forster, FG Vieira, C Meesters, S Lee, SO Twardziok, A Kanitz, J VanCampen, V Malladi, A Wilm, M Holtgrewe, S Rahmann, S Nahnsen, and J Köster. Sustainable data analysis with snakemake [version 3; peer review: 2 approved]. *F1000Research*, 10(33), 2025. doi: 10.12688/f1000research.29032.3.

[48] Jorge Nocedal and Stephen J Wright. *Numerical optimization*. Springer, 2006.

[49] Martin J Oates, David Corne, and Roger J Loader. A tri-phase multimodal evolutionary search performance profile on the'hierarchical if and only if'problem. In *GECCO*, pages 339–346, 2000.

[50] The pandas development team. pandas-dev/pandas: Pandas, February 2020. URL https://doi.org/10.5281/zenodo.3509134.

[51] Julian Perelman and contributors. jsonschema, 2012. URL https://github.com/python-jsonschema/jsonschema. Accessed: 2026.

[52] Eduardo Carvalho Pinto and Carola Doerr. Towards a more practice-aware runtime analysis of evolutionary algorithms. *arXiv preprint arXiv:1812.00493*, 2018.

[53] Sebastián Ramírez. Typer, 2019. URL https://github.com/fastapi/typer. Accessed: 2026.

[54] Jonathan Scarlett, Ilija Bogunovic, and Volkan Cevher. Lower bounds on regret for noisy gaussian process bandit optimization. In *Conference on Learning Theory*, pages 1723–1742. PMLR, 2017.

[55] Bobak Shahriari, Kevin Swersky, Ziyu Wang, Ryan P Adams, and Nando De Freitas. Taking the human out of the loop: A review of bayesian optimization. *Proceedings of the IEEE*, 104(1):148–175, 2015.

[56] Kirill Sirenkov and contributors. PyYAML, 2006. URL https://pyyaml.org. Accessed: 2026.

[57] Niranjan Srinivas, Andreas Krause, Sham M Kakade, and Matthias Seeger. Gaussian process optimization in the bandit setting: No regret and experimental design. *arXiv preprint arXiv:0912.3995*, 2009.

[58] Pauli Virtanen, Ralf Gommers, Travis E Oliphant, Matt Haberland, Tyler Reddy, David Cournapeau, Evgeni Burovski, Pearu Peterson, Warren Weckesser, Jonathan Bright, et al. Scipy 1.0: fundamental algorithms for scientific computing in python. *Nature methods*, 17(3):261–272, 2020.

[59] Richard A Watson, Gregory S Hornby, and Jordan B Pollack. Modeling building-block interdependency. In *International Conference on Parallel Problem Solving from Nature*, pages 97–106. Springer, 1998.

[60] Eric W. Weisstein. Petersen graph, n.d. URL https://mathworld.wolfram.com/PetersenGraph.html. From MathWorld—A Wolfram Resource.

[61] Carsten Witt. Tight bounds on the optimization time of a randomized search heuristic on linear functions. *Combinatorics, Probability and Computing*, 22(2): 294–318, 2013.

Appendix A

Evolutionary Algorithms, Extended

A.1 Exact Runtime Formulae

For mutation probability p, the exact expected optimization time on LeadingOnes is [11]

$$\mathbb{E}[T_p] = \frac{1}{2p^2}\big((1-p)^{-n+1} - (1-p)\big),$$

minimized at $p \approx 1.59/n$, giving $\approx 0.77n^2$. For OneMax, the sharp expansion at $p = 1/n$ is [36]

$$\mathbb{E}[T] = e\,n\ln n - 1.8925\ldots n + \tfrac{e}{2}\ln n + O(1).$$

More generally, for $p = c/n$ the expected time on any linear function is $(1 \pm o(1))\frac{e^c}{c}\,n\ln n$, minimized at $c = 1$ [61].

A.2 Implementation-Aware Variants

A practical observation of Pinto and Doerr [52] is that roughly a $1/e$ fraction of all $(1+1)$ EA iterations flip *no* bit (the offspring equals the parent), yet standard analyses charge a full function evaluation for each such iteration. Real implementations avoid these wasted evaluations by one of two strategies.

Resampling variant – $(1+1)$ EA$_{>0}$. The mutation strength ℓ is drawn from the *conditional* binomial distribution $\mathrm{Bin}_{>0}(n,p)$, i.e., $\mathrm{Bin}(n,p)$ conditioned on $\ell > 0$. This effectively skips any iteration that would flip nothing.

Shift variant – $(1+1)$ **EA**$_{0\to1}$**.** If $\ell=0$ is sampled, it is replaced by $\ell=1$, shifting all probability mass from zero flips to exactly one flip.

Both variants are analyzed in full in [52, 37]. The key runtime consequences with $p=1/n$ are summarized in Table A.1.

Algorithm	OneMax	LeadingOnes
$(1+1)$ EA	$en\ln n+O(n)\approx 2.718\,n\ln n$	$\approx 0.859\,n^2$
$(1+1)$ EA$_{>0}$	$\approx 1.718\,n\ln n+O(n)$	$\approx 0.543\,n^2$
$(1+1)$ EA$_{0\to1}$	$\approx 1.359\,n\ln n+O(n)$	$\approx 0.517\,n^2$
RLS	$nH_n\approx n\ln n$	$\approx 0.5\,n^2$

Table A.1: Expected optimization times at $p=1/n$ for the $(1+1)$ EA and its implementation-aware variants. RLS is included for reference. Bounds for the variants follow from [52, 11, 61].

Resampling or shifting yields a 35–47% reduction in function evaluations on OneMax and roughly 37–40% on LeadingOnes at $n=600$, as confirmed empirically in [52]. For the (1+1) EA$_{>0}$, the expected runtime on any linear function with $p=c/n$ becomes $(1\pm o(1))\frac{e^{c-1}}{c}\,n\ln n$ [52], a multiplicative $(1-e^{-c})$ improvement over the standard algorithm.

A.3 General Form - '$(\mu+\lambda)$-EA'

A.4 Genetic Algorithms

A.4.1 Crossover

A.5 Parameters & Operators

A.5.1 Adaptive Mutation & crossover rates

A.5.2 Selection pressure